GAONSETU APP

A Project Report
Submitted in partial fulfillment of the
Requirements for the award of the Degree of

BACHELOR OF SCIENCE(INFORMATION TECHNOLOGY)

By

Sneha Suresh Yadav TCIT76064

Under the esteemed guidance of
Dr. Ms. Manisha Nehete
Assistant Professor(BSc IT)

DEPARTMENT OF INFORMATION TECHNOLOGY
SHETH T.J. EDUCATION SOCIETY'S,
SHETH N.K.T.T. COLLEGE OF COMMERCE &
SHETH J.T.T. COLLEGE OF ARTS
(AUTONOMOUS)
THANE(W),400601
MAHARASHTRA

Academic Year(2024-25)

PROFORMA FOR THE APPROVAL PROJECT PROPOSAL

PRN No:_____ Seat No:_____ Roll no:_____

1. Name of the Student

_____

2. Title of the Project

_____

3. Name of the Guide

_____

4. Teaching Experience of Guide

_____

5. Is This your first submission ? Yes ☐ No ☐

Signature of the Student

Date: ……

Signature of the Guide

Date: …….

Signature of the Coordinator

Date: ……

UNIVERSITY OF MUMBAI
SHETH T.J. EDUCATION SOCIETY'S,
SHETH N.K.T.T. COLLEGE OF COMMERCE
&
SHETH J.T.T. COLLEGE OF ARTS,
THANE (W)(AUTONOMOUS)

DEPARTMENT OF INFORMATION TECHNOLOGY

CERTIFICATE

This is to certify that **Miss. Sneha Suresh Yadav** of T.Y.B.Sc.IT(Bachelor of Science in Information technology),Semester VI **Seat No.:TCIT76064** has undertaken & completed the project titled " **GaonSetu App**" during the academic year 2024-25 under the guidance of **Asst.Prof. Dr.Ms.Manisha Nehete** Submitted of 2025 to this college in fulfilment of the curriculum of **BACHELOR OF SCIENCE IN INFORMATION TECHNOLOGY,UNIVERSITY OF MUMBAI.**

This is to Bonafide project work and the information presented is true and original to the best of our knowledge and belief.

---	---
PROJECT GUIDE	**EXTERNAL EXAMINER**
_____	_____
Sign of Coordinator	**PRINCIPAL**

GaonSetu

ABSTRACRT

The GaonSetu App is a mobile-based platform designed to empower rural communities by providing an accessible medium for villagers to report grievances and track the resolution process. The app facilitates seamless communication between villagers and government authorities, ensuring that local issues are addressed in a timely and transparent manner. Additionally, the platform integrates a donation system, enabling philanthropic contributions from individuals or organizations to support village development projects.

The primary objective of GaonSetu is to bridge the gap between rural citizens and government services by leveraging technology to provide a structured approach to problem-solving. Villagers can log complaints about infrastructure, healthcare, education, or other community concerns, which are then reviewed and resolved by the concerned authorities. The app also provides access to government rules, regulations, and assistance schemes, ensuring that villagers are informed about their rights and benefits.

Key features include complaint tracking, notifications for complaint status, secure payment processing for donations, and real-time project updates. The app is developed using Java in Android Studio, with backend services powered by Spring Boot and data managed through MySQL. Notifications are handled through Firebase, ensuring timely updates, while integration with payment gateways facilitates safe donations.

By combining transparency, accountability, and community engagement, **GaonSetu** aims to foster a collaborative environment where rural development is streamlined, and the local populace is directly involved in improving their quality of life. Through innovative use of technology, the app seeks to ensure that village-level issues are addressed efficiently and that resources are allocated effectively, contributing to the overall betterment of rural communities.

ACKNOWLEDGEMENT

I would like to extend my heartfelt thanks to all who have helped me for completion of this project. A simple thank you would be insufficient because they are so numerous and the depth is so enormous.

I would like to acknowledge the following as being idealistic channels and fresh dimensions in the completion of this project.

I take this opportunity to thank the University of Mumbai for giving me chance to do this project.

I would like to thank my Principal, **Dr. Dilip Patil** for providing the necessary facilities required for completion of this project.

I take this opportunity to thank our SFC Coordinator, **Asst. Prof. Dr. Yogeshwari Patil** for her moral support and guidance.

I would also like to express my sincere gratitude towards my project guide **Asst. Prof.Dr. Ms. Manisha Nehete** under whose guidance and care made the project successful.

I would like to thank my College Library, for having provided various reference books and magazines related to my project.

Lastly, I would like to thank each and every person who directly or indirectly helped me in the completion of the project especially my Parents and Peers who supported me throughout my project.

Sneha Suresh Yadav(TCIT76064)

DECLARATION

I , the undersigned Miss.Sneha Suresh Yadav hereby declare that the work embodied in this project work titled **"GaonSetu App** "forms our own contribution to the research work carried out under the guidance of **Asst.Prof.Dr.Ms.Manisha Nehete** is a result of our own research work and has not been previously submitted to any other University for any other Degree to this or any other University.

Whereever reference has been made to previous works of others, it has been clearly indicated as such and included in the bibliography.

I , hereby further declare that all information of this document has been obtained and presented in accordance with academic rules and ethical conduct.

Miss. Sneha Yadav
Learner

Certified By
Asst.Prof.Dr.Ms.Manisha Nehete
Project Guide

TABLE OF CONTENTS

Title	Pg.No
Chapter 1. Indroduction	**13-21**
1.1 Backgroung	15
1.2 Objectives	17
1.3 Purpose , Scope & Applicability	19
1.3.1 Purpose	19
1.3.2 Scope	19
1.3.3 Applicability	19
1.4 Achievements	20
1.5 Organisation of Report	20
Chapter 2. Survey of Technologies	**23-26**
Chapter 3. Requirements andAnalysis	**28-44**
3.1 Problem Definition	28
3.2 Requirements Specification	29
3.3 Planning and Scheduling	31
3.4 Software and Hardware Requirements	33
3.5 Preliminary Product Description	34
3.6 Conceptual Models	35
Chapter 4. System Design	**46-60**
4.1 Basic Modules	46
4.2 Data Design	47

4.2.1 Schema Design 47

4.2.2 Data Integrity and Constraints 48

4.3 Procedural Design 50

4.3.1 Logic Diagrams 50
4.3.2 Data Structures 51

4.3.3 Algorithms Design 53
4.4 User interface design 55
4.5 Security Issues 58
4.6 Test Cases Design 60

Chapter 5. Implementation and Testing 62-118

5.1 Implementation Approaches 62
5.2 Coding Details and Code Efficiency 63
5.2.1 Code Efficiency 111
5.3 Testing Approach 112
5.3.1 Unit Testing 113
5.3.2 Integrated Testing 115
5.3.3 Beta Testing 115
5.4 Modifications and Improvements 116
5.6 Test Case 118

Chapter 6. Results and Discussion 120-124

6.1 Test Reports 120
6.2 User Documentation 121

Chapter 7. Conclusion 126-128

7.1 Conclusion 126
7.1.1 Significance of the System 126
7.2 Limitations of the System 127
7.3 Future Scope of the Project 128

Chapter 8. Refrences 129

LIST OF TABLES

Table No.	Table Name	Page No.
3.6.1	Use Case Table	35
3.6.2	Activity Diagram Table	40
3.6.3	Sequence Diagram Table	43
4.2.2.1	User Table	48
4.2.2.2	Complaint Table	48
4.2.2.3	Donation Table	48
4.2.2.4	Notification Table	49
4.2.2.5	Government Rules (GR)Table	49
4.6.1	Test Case Design Table	60
5.6.1	Test Case Table	118
6.1.1	Test Report Table	120

LIST OF FIGURES

Figure No.	Figure Name	Page No.
2.1	Android Studio	24
2.2	Java	25
2.3	MySQL Database	25
2.4	Payment Gateway	26
3.3.1	Gantt Chart	31
3.3.2	Incremental Model	32
3.6.1	Use-Case Diagram	36
3.6.2	Data Flow Diagram Level-0	38
3.6.3	Data Flow Diagram Level-1	39
3.6.4	Activity Diagram-1	42
3.6.5	Activity Diagram-2	42
3.6.6	Sequence Diagram	44
4.2.1.1	Schema Design	47
5.3	Testing Gantt Chart	112
6.2.1	Splash Screen	121
6.2.2	Language Screen	121
6.2.3	User Registration Page Screen	122
6.2.4	User Login Page Screen	122
6.2.5	Home Page Screen	123
6.2.6	Complain Page Screen	123
6.2.7	Profile Page Screen	124
6.2.8	Admin Page Screen	124

CHAPTER 1
INTRODUCTION

INTRODUCTION

In India, the majority of the population resides in rural areas where access to basic infrastructure and services can often be limited. Villages face numerous challenges such as poor road connectivity, inadequate healthcare, erratic electricity supply, water shortages, and lack of access to government services. Many of these problems persist due to a communication gap between villagers, local authorities, and organizations that aim to address these issues. This is where GaonSetu comes in—a comprehensive mobile application designed specifically to empower and uplift rural communities by acting as a digital bridge that connects villagers with essential services and solutions.

The name "GaonSetu" is derived from two words—'Gaon', meaning village, and 'Setu', meaning bridge. Together, it signifies a platform that creates a bridge between villages and the necessary resources they need for development, growth, and problem-solving. GaonSetu simplifies the process of reporting issues, accessing governmental and non-governmental services, and improving communication between rural residents and service providers.

The app is designed to be user-friendly and intuitive, even for individuals with limited experience in using technology. With its multilingual interface and simple layout, GaonSetu caters to a wide spectrum of rural users, enabling them to access critical services with just a few taps on their smartphones. Villagers can use the app to report issues like broken roads, water supply disruptions, healthcare emergencies, and more. These issues are then communicated to the relevant authorities or service providers in real-time, ensuring prompt action and transparency.

One of the standout features of GaonSetu is its integration with NGOs and government services. By establishing partnerships with non-governmental organizations government bodies, GaonSetu facilitates direct access to welfare programs, healthcare initiatives, education schemes, and agricultural support. This allows villagers to stay informed about opportunities for development and benefit from initiatives that are often difficult to navigate without technological intervention.

Additionally, GaonSetu includes a donation and fundraising system where users

and external supporters can contribute to specific village projects or emergencies. Whether it's raising funds for building a local school or providing relief during a natural disaster, the app empowers communities to take collective action for the betterment of their village.

Through its holistic approach, GaonSetu aims to improve the quality of life in rural areas by streamlining access to solutions, fostering collaboration between communities and service providers, and promoting self-reliance. It’s not just an app—it’s a movement towards creating connected, empowered, and thriving villages across the country.

In summary, GaonSetu is designed to bring rural communities closer to essential services, bridging the gap between traditional rural life and modern solutions. By empowering villagers with an easy-to-use digital platform, GaonSetu is poised to drive rural development and make villages more self-sufficient, connected, and progressive.

1.1 Background

Rural India forms the backbone of the country's population, with over 65% of Indians and residing in villages. Despite their integral role in the nation's economy, especially in sectors like agriculture and small-scale industries, villages have long struggled with inadequate infrastructure, poor access to healthcare, erratic water and electricity supply, limited educational facilities. This results in a significant development gap between urban and rural areas.

The Indian government and various non-governmental organizations (NGOs) have implemented numerous schemes and programs to uplift rural communities. However, many of these programs remain underutilized or poorly executed due to several factors:

1. **Communication Barriers:**
   Villagers often face challenges in effectively communicating their needs or issues to the authorities responsible for resolving them. Traditional methods of communication are slow and inefficient.
2. **Lack of Awareness**:
   Many rural residents are unaware of the governmental schemes and services available to them due to poor information dissemination. The absence of a centralized, accessible platform makes it difficult for people to access essential information.
3. **Limited Digital Connectivity**:
   Although digital infrastructure has made significant progress, many villages still face limited access to technology, further widening the gap between rural and urban areas.
4. **Transparency and Accountability**:
   Even when issues are reported, the lack of real-time updates and transparency in the system leads to delays in resolving problems, causing frustration and mistrust among Villagers.

Recognizing these challenges, GaonSetu was conceptualized to address these specific issues. The app acts as a digital bridge between rural communities and service providers, helping to modernize communication and problem-solving processes. With the rise of smartphone penetration in rural India, GaonSetu leverages technology to ensure that even the most remote villages have access to the resources they need for development.

The Indian government's push towards Digital India has also emphasized the need for greater inclusion of rural communities in the digital revolution. Rural areas are slowly adopting smartphones and internet usage, creating the perfect opportunity for a mobile application that can empower villagers. The JAM trinity (Jan Dhan-Aadhaar-Mobile) has already laid the foundation for digitization, and apps like GaonSetu aim to build upon this by offering villagers direct access to critical services through a simple, user-friendly platform.

Inspiration for GaonSetu comes from observing the success of digital governance platforms in urban settings, where citizens can easily file complaints, track issues, and access services online. However, rural areas require a tailored solution—one that is built with the specific needs and challenges of villages in mind. GaonSetu incorporates elements of multilingual support, offline capabilities, and local-level service access, making it accessible even to those with limited technical knowledge or internet availability.

In addition to solving immediate local problems, the app fosters greater community participation in development initiatives. It provides a platform for villagers to collectively raise funds or seek donations for community projects, such as building schools, improving sanitation, or responding to natural disasters. This ability to unite the community around common goals enhances social cohesion and collective action, both critical for long-term development.

Moreover, NGOs and government bodies benefit from the app's ability to streamline. service delivery and reach underserved populations. GaonSetu acts as a digital facilitator, ensuring that available schemes and services reach the intended beneficiaries effectively. This reduces the bureaucratic burden and enhances the overall efficiency of rural development initiatives

1.2 Objectives

The GaonSetu app aims to address the unique challenges faced by rural communities by providing them with a digital platform that connects them to essential services, authorities, and resources. The core objectives of the app are as follows:

1. **Facilitate Problem Reporting and Resolution**:
   - Provide villagers with an easy-to-use platform to report local issues such as infrastructure problems (roads, water, electricity), healthcare needs, and other essential services.
   - Ensure that complaints and concerns are quickly communicated to the relevant authorities or service providers for timely resolution.
2. **Empower Rural Communities**:
   - Empower villagers by offering them a tool to actively participate in their own development, giving them a voice to communicate their needs and concerns to local authorities, NGOs, and government bodies.
   - Promote **self-reliance** by creating a space for villagers to collaborate on community-driven initiatives such as education, sanitation, and health.
3. **Increase Awareness and Access to Government Schemes**:
   - Bridge the information gap by providing villagers with accurate and up-to-date information on government welfare schemes, programs, and subsidies that they may be eligible for.
   - Facilitate easy access to these schemes through the app, reducing the complexity and time taken to benefit from government initiatives.
4. **Connect Villages with NGOs and Charitable Organizations**:
   - Establish a channel through which NGOs and charitable organizations can interact directly with rural communities, offering services such as education, healthcare, agricultural support, and more.
   - Create opportunities for villages to receive donations and aid from individuals, organizations, and corporations for projects and community development initiatives.
5. **Promote Transparency and Accountability**:
   - Enable real-time tracking of issues reported through the app, providing transparency on the status of complaints and solutions.
   - Build a feedback loop where villagers can monitor the progress of their issues, holding local authorities accountable for their actions.
6. **Encourage Community Participation**:

- Encourage collective action by giving villagers a platform to initiate, support, and contribute to community development efforts such as building schools, improving water supply, or disaster relief initiatives.

7. Leverage Digital Infrastructure for Rural Development:

- Simplify the use of digital services by providing a user-friendly, multilingual interface that is accessible even to individuals with limited technical knowledge.

8. Support Sustainable Development:

- Contribute to the long-term development of villages by connecting them with sustainable solutions, whether it be in agriculture, renewable energy, or infrastructure.

9. Facilitate Donations and Fundraising for Community Projects:

- Provide a mechanism for raising funds for community-driven projects or emergency relief efforts, enabling villagers to crowdsource financial support for local development.

1.3. Purpose, Scope, and Applicability

1.3.1 Purpose

The GaonSetu app is developed with the purpose of empowering rural communities by providing them with a digital platform that bridges the gap between their needs and essential services. The app aims to create a seamless channel for communication, problem-solving, and access to resources, addressing the unique challenges faced by villages in India. Its goal is to create an inclusive platform that ensures villagers have the resources, transparency, and support they need to address local challenges and foster sustainable growth in rural India.

1.3.2 Scope

The GaonSetu app is designed with a broad scope that addresses various aspects of rural development, empowering villagers through digital tools to report issues, access essential services, and drive local improvements. The app's reach extends beyond individual problem-solving to fostering community collaboration, improving service delivery, and enhancing communication between rural populations and authorities.

1.3.3 Applicability

GaonSetu is applicable in various domains such as rural development by strengthening digital access in villages, agriculture by providing valuable insights and real-time market prices, healthcare by connecting users with medical facilities and online consultations, education by offering online learning resources and skill development courses, e-commerce by empowering rural artisans and small businesses to reach wider markets, and government services by ensuring easier access to welfare schemes and applications.

1.4 Achievements

GaonSetu has successfully facilitated digital inclusion in multiple villages, improving access to essential government schemes and benefits. It has empowered rural businesses by providing an e-commerce platform, leading to increased income opportunities for artisans and small entrepreneurs. The app has also helped farmers make informed decisions through real-time agricultural data and weather forecasts. Additionally, it has enhanced healthcare accessibility by connecting users with telemedicine services and nearby medical facilities. With its user-friendly multi-language support, GaonSetu has ensured inclusivity, allowing a broader population to benefit from digital advancements.

1.5 Organisation of Report

The report on GaonSetu: Bridging Villages to Modern Services is organized as follows:

1. **Introduction:**
   This section introduces the GaonSetu app, highlighting the problems faced by villagers and explaining how the app addresses these challenges. It outlines the project's objectives and the scope of implementation.
2. **System Analysis and Requirements:**
   This section analyzes the existing systems, their limitations, and proposes the GaonSetu app as a solution. It also defines the functional and non-functional requirements of the application.
3. **System Design and Architecture:**
   This section covers the architectural design of the app, including system architecture diagrams, use case diagrams, and database design. It explains how different modules interact with each other.
4. **App Development and Implementation:**
   This section describes the tools and technologies used in the development of the app. It explains key modules such as language selection, OTP-based login, and the admin panel.
5. **User Interface and Experience:**
   This section presents the user interface design, including screenshots of key screens and descriptions of the app's features, ensuring simplicity and usability.
6. **Testing and Validation:**
   This section discusses the testing methodologies applied, including unit testing, integration testing, and user acceptance testing. It presents test cases and results.

7. **Results and Impact:**

   This section highlights the expected outcomes of the app, including its impact on villagers and possible limitations. It also discusses the future scope for enhancing the application.

8. **Conclusion and References:**

   The final section summarizes the project's achievements and provides references to the resources used during the development process.

CHAPTER 2

SURVEY OF TECHNOLOGIES

SURVEY OF TECHNOLOGIES

Developing a mobile application, especially one aimed at empowering villages like GaonSetu, is similar to building a bridge between modern technology and rural communities. Before initiating the development process, it is essential to analyze the application's requirements, choose appropriate technologies, and ensure that the platform is secure, user-friendly, and efficient.

The most critical step in developing a mobile app is selecting the right programming language and development environment. Since GaonSetu is being developed as a native Android application, Java has been chosen as the primary programming language, supported by Android Studio for an efficient development workflow. Java is widely used for Android apps due to its stability, security, and strong support for object-oriented programming.

After writing the core logic of the application, testing the app on a real or virtual device becomes necessary. Even for an expert developer, unexpected bugs may occur, so server-side coding is required to ensure smooth backend communication and seamless data management.

In this context, the backend of GaonSetu requires a robust and secure platform to handle user authentication, language preferences, and manage data effectively. PHP has been chosen as the backend language, integrated with a MySQL database to store and manage data. This combination offers ease of maintenance and ensures a smooth connection between the frontend and backend. RESTful APIs will be used to establish secure communication between the mobile app and the server.

Additionally, since the app will be used by villagers with limited technological exposure, the user interface should be intuitive and easy to navigate. The language selection screen will allow users to choose their preferred language using leaf-shaped buttons with checkboxes, which is aligned with the theme of village culture. The selected language will be stored using SharedPreferences and can be changed later from the settings.

To enhance user engagement, GaonSetu will support push notifications using Firebase Cloud Messaging (FCM) to keep users informed about important updates and notifications. For future scalability, analytical tools such as

Firebase Analytics can be integrated to analyze user behavior and app performance, enabling continuous improvement.

In conclusion, GaonSetu's technology stack, which includes Java for the frontend, PHP and MySQL for the backend, and SharedPreferences for language storage, has been carefully chosen to meet the needs of rural communities. Future enhancements may include AI-based recommendations and real-time data analytics to further empower villagers and streamline their access to essential services.

Why should we use Android Studio?

Android Studio is the official Integrated Development Environment (IDE) for developing Android applications, making it the ideal choice for building the GaonSetu app. Since the app is being developed in Java, Android Studio provides seamless support for Java and Android APIs, ensuring smooth coding and debugging. It offers a rich user interface design tool that allows the creation of responsive and dynamic layouts, ensuring the app looks appealing and functions well on different devices. The built-in Android Emulator makes it easy to test and debug the app in real-time without requiring a physical device.

Figure 1: Android Studio

Android Studio simplifies the integration of APIs and external libraries, which is essential for connecting GaonSetu's backend, managing user authentication, and enabling push notifications through Firebase Cloud Messaging (FCM). It also supports version control using GitHub, making it easier to manage and track project versions. Advanced debugging and performance analysis tools, such as Logcat and Profiler, help identify and fix errors efficiently. Additionally, Android Studio automates the build and deployment process through its Gradle build system, ensuring quick and smooth compilation of the app.

Why should we use Java Programming Language ?

Java is the preferred language for developing the GaonSetu app because it is the official and most widely used language for Android development. It offers strong support for object-oriented programming (OOP), making the code more modular, scalable, and easier to maintain. Java's stability and vast library ecosystem simplify the implementation of essential features, such as database integration, user authentication, and API communication.

Figure 2: Java

Moreover, Java provides excellent security features, which are crucial for handling sensitive user data in the GaonSetu app. Its compatibility with Android Studio ensures seamless development, debugging, and testing. Java also enables efficient memory management and enhances app performance, ensuring a smooth and responsive user experience. These benefits make Java the ideal choice for building a secure, reliable, and scalable application like GaonSetu.

Why we need MySQL Database ?

MySQL is used as the database for the GaonSetu app because it is a reliable, scalable, and secure relational database management system (RDBMS). It efficiently stores and manages structured data, such as user profiles, language preferences, and activity logs, ensuring smooth app functionality. Since MySQL supports SQL queries, it allows easy data retrieval and manipulation, enabling the app to perform operations like user authentication, storing preferences, and managing updates seamlessly.

Figure 3: MySQL Database

Additionally, MySQL offers robust data security features, ensuring that sensitive user information is protected. Its ability to handle large volumes of data efficiently makes it suitable for GaonSetu, which is expected to manage information from multiple users over time. The combination of reliability, speed, and security makes MySQL an ideal choice for ensuring smooth backend operations and maintaining data integrity in GaonSetu.

Why Should We Use a Payment Gateway ?

Integrating a payment gateway in the GaonSetu app is essential to facilitate secure and seamless online transactions for services or product purchases. A payment gateway acts as a

bridge between the app and financial institutions, ensuring that payments are processed securely by encrypting sensitive information such as credit/debit card details and UPI credentials. It enables users to make payments effortlessly, boosting convenience and trust.

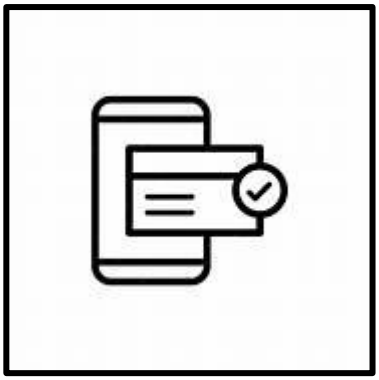


Figure 3: Payment Gateway

Moreover, a payment gateway automates the payment process, reducing the chances of errors and ensuring timely transactions. It supports multiple payment methods, including UPI, digital wallets, and net banking, catering to a diverse user base. By enabling secure payments, the payment gateway enhances user confidence and ensures that financial transactions within the GaonSetu app are efficient, transparent, and compliant with industry standards.

CHAPTER 3

REQUIREMENTS AND ANALYSIS

REQUIREMENTS AND ANALYSIS

3.1 Problem Definition

In many rural areas, villagers face challenges in accessing essential services, information, and resources that could improve their quality of life. There is often a lack of awareness about government schemes, agricultural practices, healthcare facilities, and educational opportunities. Additionally, communication between village authorities and residents is limited, making it difficult to address local issues effectively.

Due to this gap, villagers are unable to fully utilize available resources and remain disconnected from modern advancements.

GaonSetu aims to bridge this gap by providing a digital platform that connects villagers with essential services, information, and resources. The app will serve as a one-stop solution for accessing government schemes, agricultural advice, healthcare updates, educational opportunities, and local news. It will also facilitate better communication between local authorities and residents, empowering the community with knowledge and opportunities.

In addition to addressing the lack of information and services, **GaonSetu** also aims to promote digital literacy and inclusivity by providing an intuitive and easy-to-use interface designed specifically for rural users. Since many villagers may not be familiar with complex mobile applications, the app ensures simplicity in navigation and usability, similar to government apps, to make the transition to digital services seamless. The language selection feature, with options displayed inside leaf-shaped buttons, allows users to choose their preferred language, ensuring that the app caters to diverse linguistic backgrounds.

Moreover, the inclusion of **OTP-based login and sign-up** ensures secure and hassle-free access, enhancing trust and encouraging more villagers to engage with digital platforms. By empowering communities with knowledge, resources, and digital tools, **GaonSetu** fosters self-reliance and paves the way for a more connected and informed rural population.

3.2 Requirement Specification

The requirement analysis for the GaonSetu app aims to define the key. functionalities, user needs, system behavior, and external interfaces necessary to deliver a seamless experience to villagers for resolving complaints and accessing services.

Functional Requirements:

1. **User Registration and Authentication**
   - OTP-based login and sign-up to verify users securely.
   - Simple and intuitive registration process designed for villagers.
2. **Language Selection and Management**
   - Display all Indian languages on the initial screen in leaf-shaped buttons with checkboxes.
   - Allow users to select their preferred language, which is stored and applied using SharedPreferences.
   - Language selection should appear only once during the first app installation, with an option to change language later from the settings.
3. **Information and Service Access**
   - Provide categorized information about government schemes, agricultural practices, healthcare services, and educational resources.
   - Enable real-time updates and notifications for new schemes and local announcements.
4. **Admin Panel for Management**
   - A mobile admin app to manage and monitor user data, schemes, and updates.
   - Ability to handle language preferences, verify OTPs, and manage content.

Non-Functional Requirements:

1. **Performance**
   - Fast response time, ensuring that data retrieval and updates are efficient.
   - Support for low-bandwidth internet connectivity in rural areas.
2. **Scalability**
   - Ability to handle an increasing number of users and data as the app grows.
3. **Security**
   - Secure data storage using encryption.
   - OTP-based authentication to prevent unauthorized access.
4. **Usability**
   - Simple and easy-to-use interface similar to government apps.
   - Minimal learning curve to ensure accessibility for less tech-savvy users.
5. **Reliability**

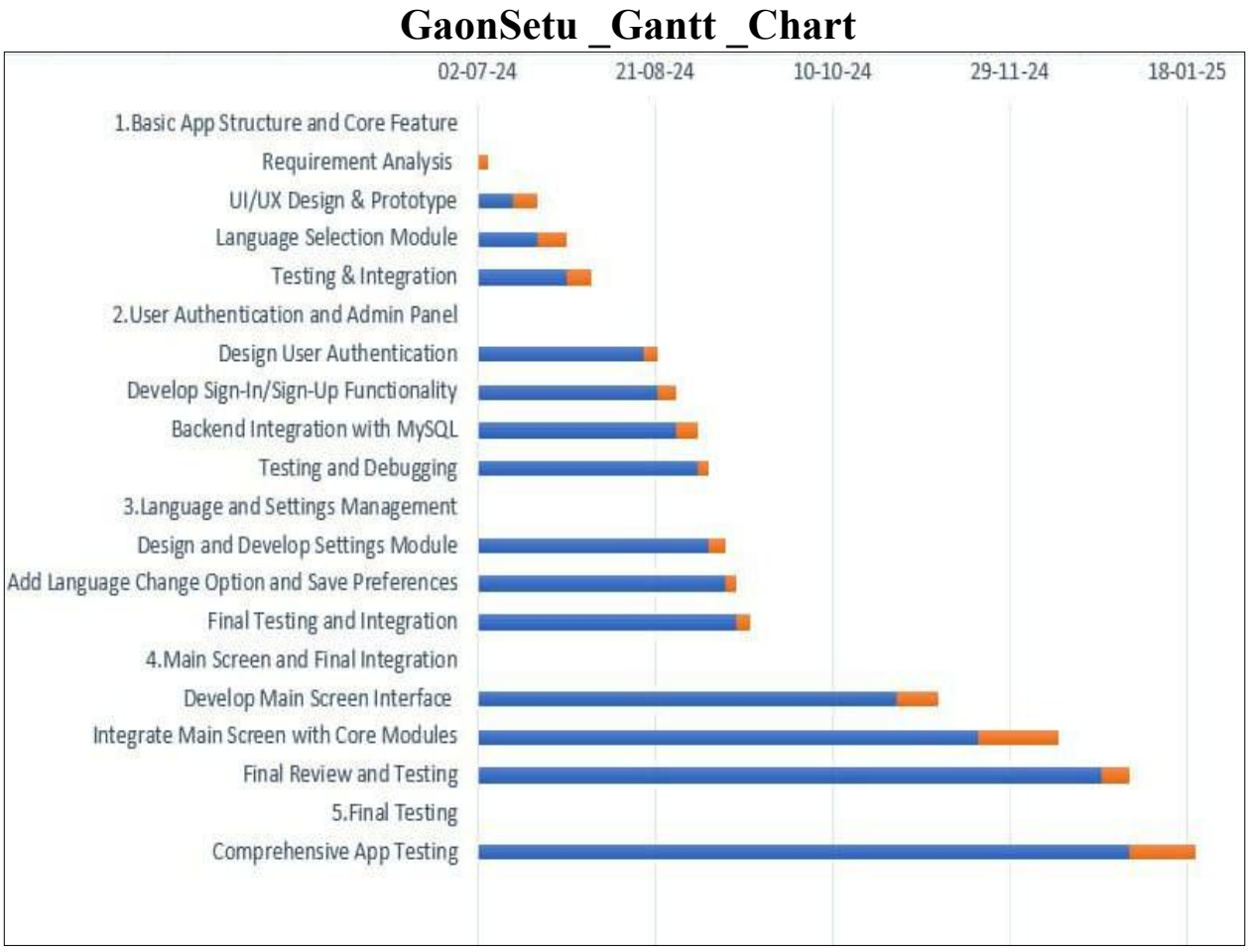
- Ensure 24/7 availability with minimal downtime.
- Robust backend integration with MySQL to handle data efficiently.

3.3 Planning and Scheduling

The development of the GaonSetu app follows a well-structured **planning and scheduling process** to ensure timely completion and efficient resource management. A **Gantt chart** is used to visualize the project timeline by displaying tasks on the vertical axis and time intervals on the horizontal axis. It highlights the start and finish dates of each task and shows the dependencies between different modules, allowing for a clear understanding of the project flow.

The development process is divided into multiple increments, where essential phases such as **requirement analysis, UI/UX design, backend integration, module development, and testing** are carried out sequentially. The Gantt chart uses an **early start time approach**, ensuring that each task begins immediately after its prerequisites are complete. This minimizes idle time and optimizes the overall workflow.

Additionally, the Gantt chart helps in scheduling resources effectively and tracking progress to identify any delays or bottlenecks. Regular monitoring and adjustments ensure that the project stays aligned with its objectives. This systematic approach ensures that the GaonSetu app is developed with all desired features, delivered on time, and ready to empower villages with innovative solutions.



Gantt Chart

Software Development Stages:

The Incremental Model was followed to develop and implement the **GaonSetu App** in multiple builds (increments), with each increment adding new functionality.

- Increment 1: Basic Registration and Language Selection:
  - OTP-based login and sign-up for secure user verification.

Language selection screen with leaf-shaped buttons for choosing preferred language, stored using SharedPreferences.

- **Increment 2: Information and Scheme Access:**
  - Module to access government schemes, healthcare information, and agricultural practices.

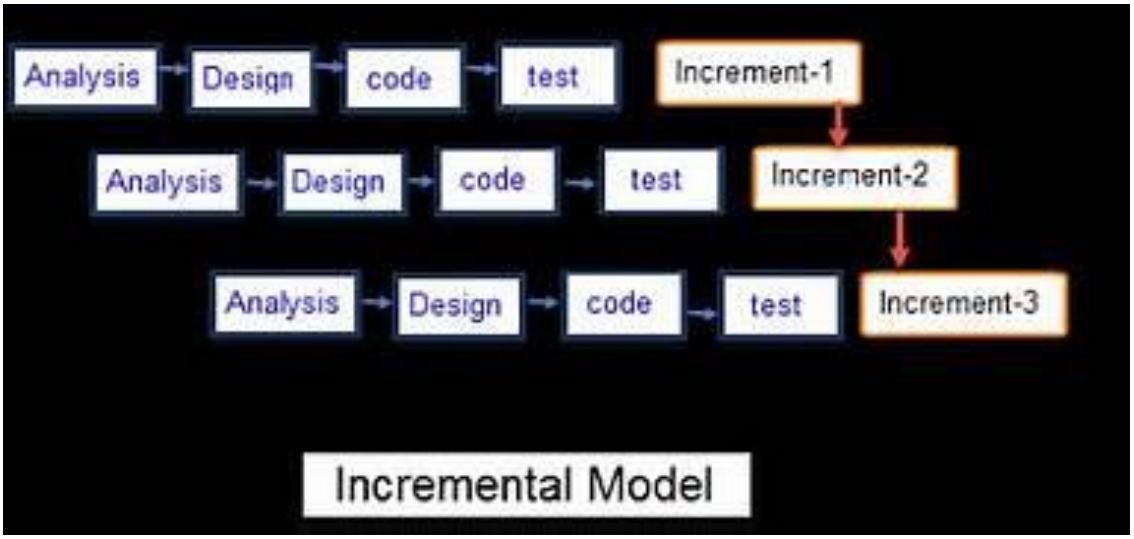
Search and filter features to improve information discovery.

- **Increment 3: Communication Module:**
  - Grievance submission and communication platform between villagers and local authorities.

Notification system for important updates**.**

- **Increment 4: Admin Panel Development**

Mobile admin app to manage user data, verify OTPs, and update relevant content.



Why Incremental Model is Suitable for GaonSetu?

- Allows gradual development and deployment, ensuring user feedback is incorporated in future increments.
- Reduces initial risk by delivering a basic working version early.
- Enables continuous improvement and feature expansion over time.

3.4 Software and Hardware Requirements

Hardware Requirements:

1. **For Development**
    - Processor: Minimum Intel i5 (or equivalent)
    - RAM: Minimum 8 GB (Recommended 16 GB for smooth Android Studio performance)
    - Storage: Minimum 256 GB SSD (Recommended 512 GB SSD for faster build and deployment)
    - Operating System:
        - Windows 10/11 (64-bit)
        - macOS 12+ (for Mac users)
2. **For Backend Server**
    - Processor: Intel Xeon or equivalent
    - RAM: Minimum 8 GB (Recommended 16 GB for handling multiple requests)
    - Storage: Minimum 200 GB HDD or SSD
    - Operating System: Linux/Ubuntu for server hosting
    - Web Server: Apache or Nginx
3. **For Testing and Deployment**
    - Android Device: Minimum Android 5.0 (Lollipop) and above
    - Recommended Devices: Budget smartphones with low processing capacity to simulate rural user experiences

Software Requirements:

Frontend Development

- IDE: Android Studio (Latest Version)
- Programming Language: Java
- Minimum Android SDK Version: 21 (Lollipop)
- Target Android SDK Version: 33 (Android 13)

Backend Development

- Programming Language: PHP
- Database: MySQL

Database Management

- DBMS: MySQL
- Database Tool: MySQL Workbench

Admin Panel Development

- Platform: Android Studio (for mobile-based admin app)
- Backend Integration: PHP with MySQL

3.5 Preliminary Product Description

GaonSetu is a mobile application designed to empower rural communities by providing easy access to essential information, government services, and resources. The app serves as a **digital bridge** between villagers and various service providers, addressing the digital divide that often limits rural development.

The app is being developed using the **Incremental Model**, where core features are introduced in phases to ensure smooth functionality and user adoption.

In its initial phase, **GaonSetu** focuses on:

- Simplifying user registration and language selection.
- Offering relevant government scheme information.
- Providing agricultural and healthcare guidance.

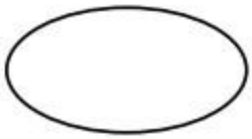
Subsequent increments will introduce advanced features such as grievance submission, communication with authorities, and an admin panel to manage user interactions. The platform is designed to be user-friendly, multilingual, and highly accessible, ensuring that even users with limited digital literacy can easily navigate and benefit from the app.

3.6 Conceptual Module

1. USE CASE DIAGRAMS:

A use case diagram is a graphical depiction of a user's possible interactions with a system. It captures the system's functionality and requirements by incorporating use cases, actors, and their relationships. A use case represents a distinct functionality or goal of a system. An actor represents a user or another system that interacts with the system. A use case diagram is useful for visualizing the dynamic behavior of a system.

COMPONENTS OF USE CASE DIAGRAMS:

Components	Description
Actor	Actors are the users of system and are represented using stick figures
Use case 	Use cases are represented using ellipses. The use cases are labelled with verbs that represent the system functions
System	System boundaries are represented using a rectangle that contains use cases. Actors are placed outside system boundaries.

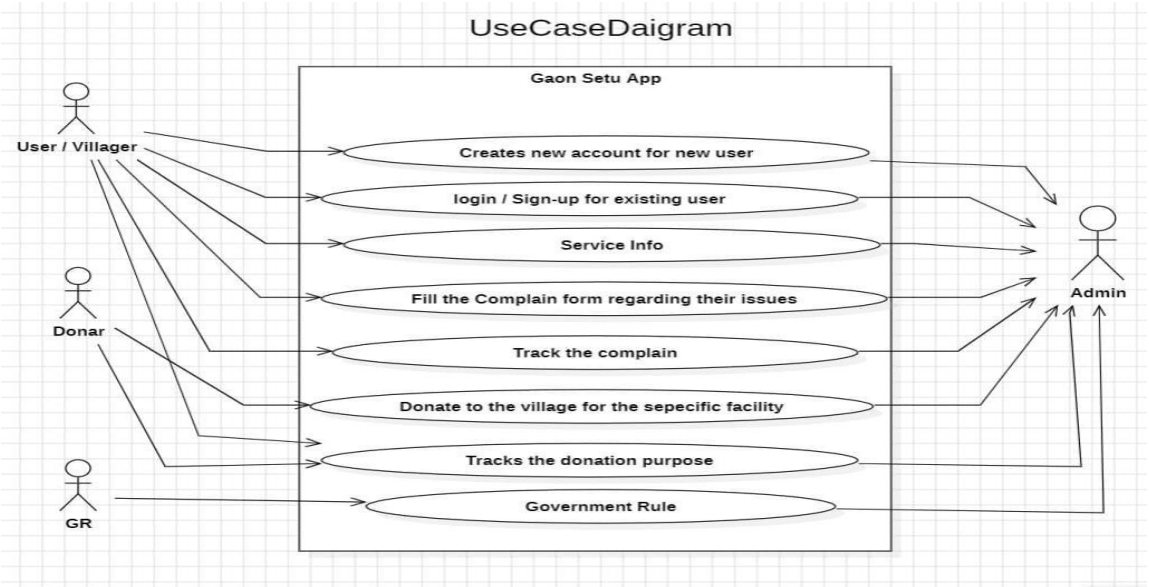
USE OF USE-CASE DIAGRAM:

A use case diagram is a graphical representation used in software engineering to illustrate how a system interacts with its actors (users, external systems, or other components) to achieve specific goals or tasks. It is a fundamental tool in the Unified Modelling Language (UML) and serves several important purposes in the software development process:

1. **Requirements Analysis:** Use case diagrams are often created during the early stages of software development to help gather and clarify requirements. They provide a highlevel view of the system's functionality from the perspective of its users, helping to identify what the system should do.

2. **Communication:** Use case diagrams are an effective means of communication between stakeholders, including developers, designers, project managers, and end users. They provide a visual representation of the system's behavior and help ensure that everyone has a common understanding of the system's functionality.

System Design: Use case diagrams serve as a foundation for system design. They help in determining which components or modules are necessary to implement each use case. This, in turn, aids in architectural decisions and the allocation of responsibilities among different parts of the system



Use-Case Diagram

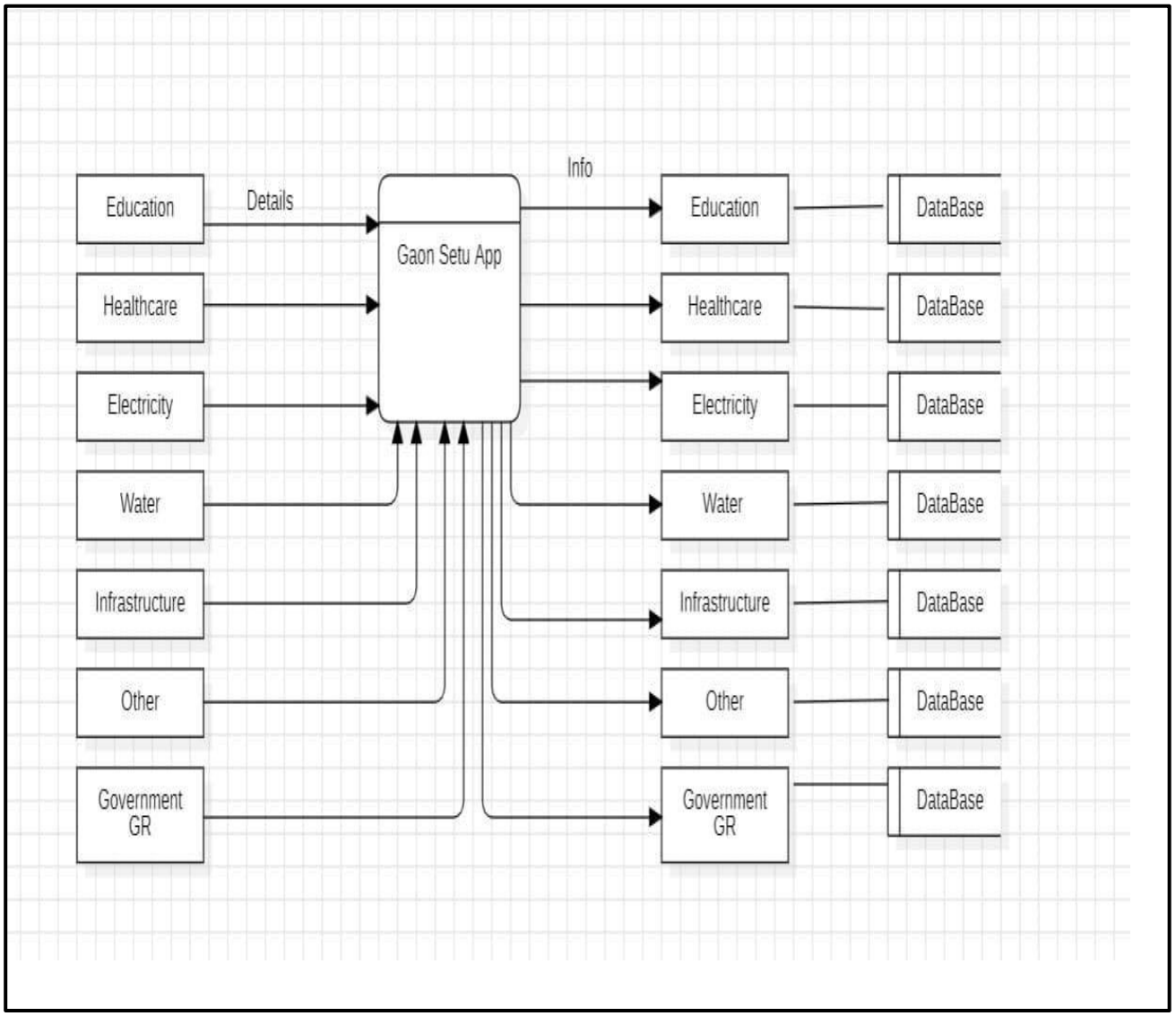
2. DATA FLOW DIAGRAMS(DFD):

A data flow diagram is a graphical representation used in software engineering and systems analysis to depict how data flows within a system or between various components of a system. It provides a visual way to illustrate the movement and transformation of data as it moves through processes, data stores, and external entities. DFDs are particularly useful for understanding, analyzing, and designing information systems.

USES OF DATA FLOW DIAGRAM:

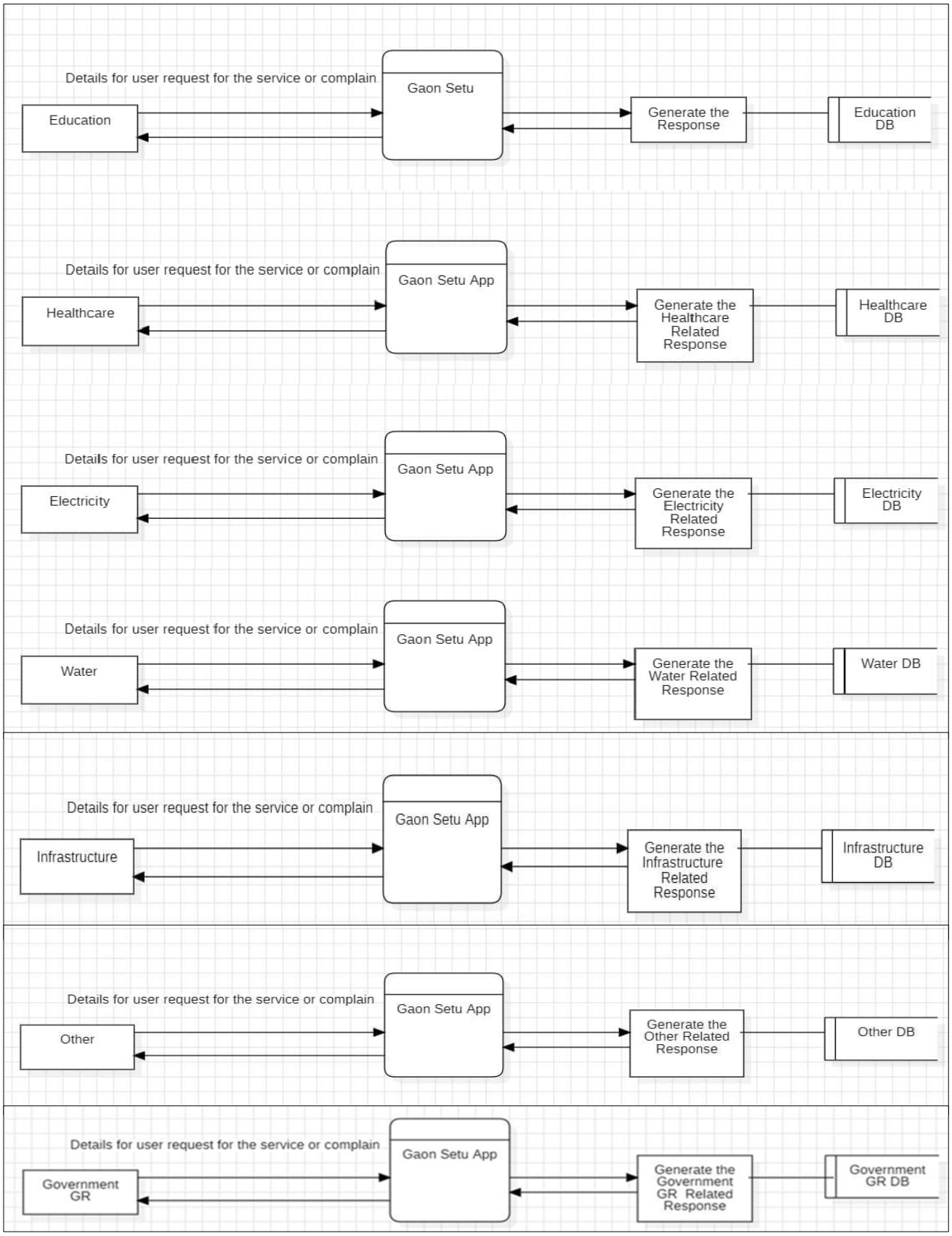
1. **Documentation:** DFDs are valuable documentation tools for both current and future reference. They document the system's data flow, which can be helpful for maintenance, troubleshooting, and system evolution.
2. **Testing and Validation:** DFDs can be used to generate test cases to validate that the system behaves as expected. By understanding the data flow, testers can create test scenarios to cover various aspects of system functionality.
3. **Identifying Redundancy and Inefficiencies:** DFDs can reveal redundant data flows or inefficient processes within a system. This information can be used to optimize system performance and resource utilization.
4. **Security Analysis:** DFDs can aid in identifying potential security vulnerabilities or data leaks by highlighting data flows that cross system boundaries or interact with external entities.
5. **Process Improvement:** In business process reengineering and improvement efforts, DFDs are used to model existing processes, analyze inefficiencies, and design improved workflows.
6. **Scalability Planning:** DFDs can help in planning for system scalability by showing how data flows might need to change as the system grows or evolves.
7. **Data Modelling:** DFDs can serve as a starting point for developing data models, including entity-relationship diagrams, which help define the structure of the data within the system.

Context Level Diagram Level-0:



Data Flow Diagram Level-0

Context Level Diagram Level-1:

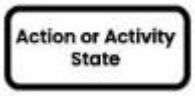
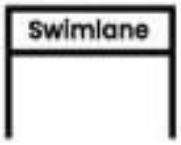


Data Flow Diagram Level-1

3. **ACTIVITY DIAGRAMS:**

Activity diagrams show the steps involved in how a system works, helping us understand the flow of control. They display the order in which activities happen and whether they occur one after the other (sequential) or at the same time (concurrent). These diagrams help explain what triggers certain actions or events in a system.

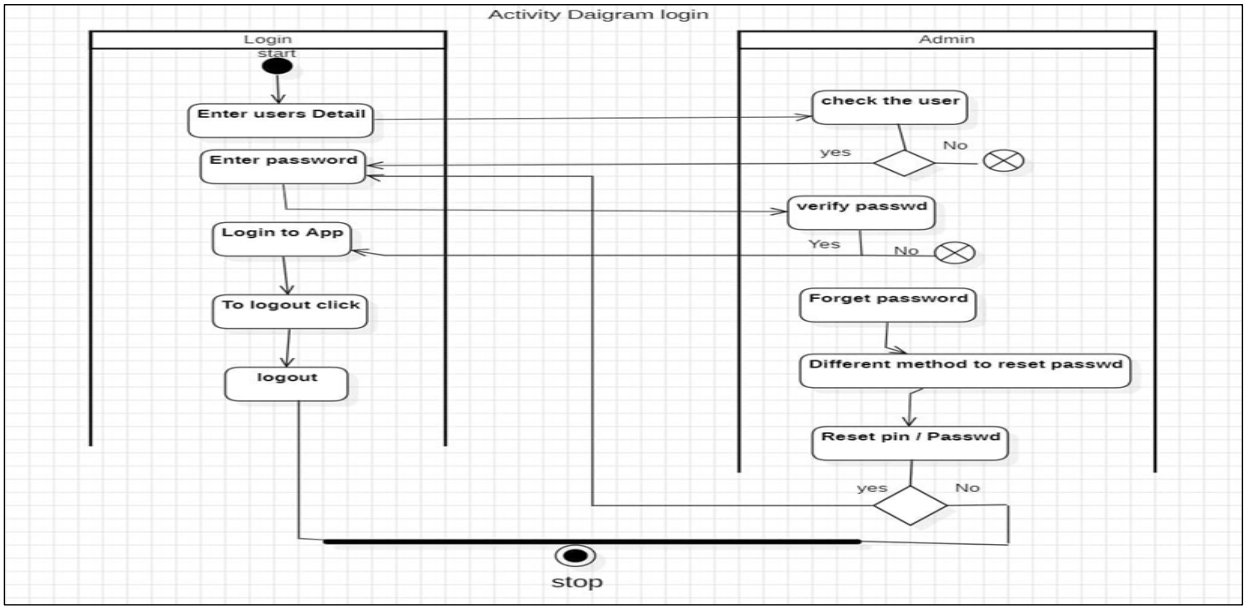
NOTATIONS OF ACTIVITY DIAGRAM:

Notation	Description
Start	Shows the beginning of a process.
Decision	Represents a decision
Action or Activity State 	Represents execution of an action on objects or by objects.
Action Flow or Control flows	Used to show the transition from one activity state to another activity state.
Swimlanes 	Used to add modularity to the activity diagram.
Flow final symbol	Represents the end of a specific process flow

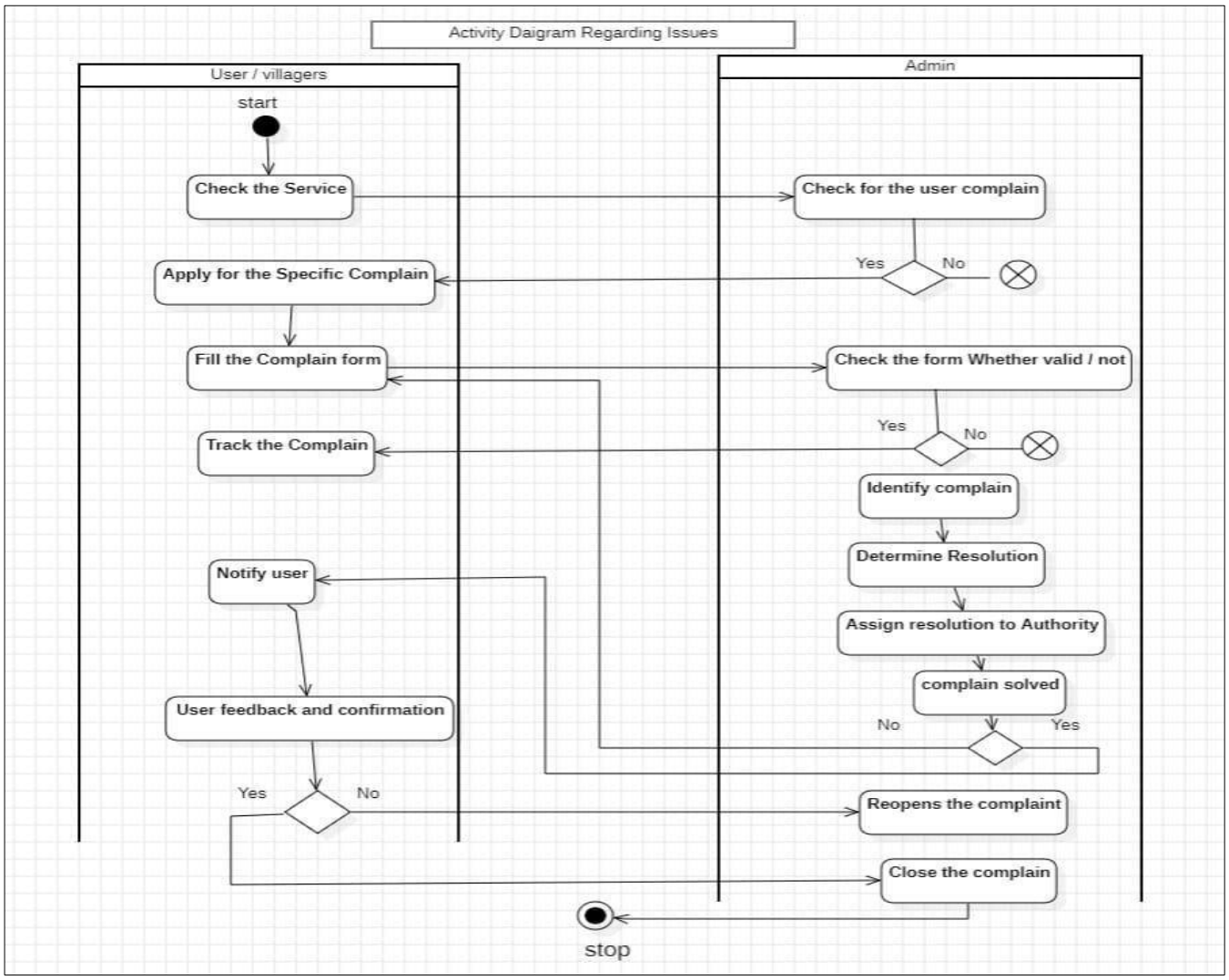
Final State	A system or a process can have multiple final states.

USE OF ACTIVITY DIAGRAM:

1. **Modeling Workflows or Processes**: When you need to map out a business process, workflow, or the steps involved in a use case, activity diagrams help visualize the flow of activities.
2. **Concurrent or Parallel Processing**: If your system or process involves activities happening simultaneously, an activity diagram can clearly show the parallel flow of tasks.
3. **Understanding the Dynamic Behavior**: When it's essential to depict how a system changes over time and moves between different states based on events or conditions, activity diagrams are effective.
4. **Clarifying Complex Logic**: Use an activity diagram to simplify complex decision-making processes with branching paths and different outcomes.
5. **System Design and Analysis**: During the design phase of a software system, activity diagrams help developers and stakeholders understand how different parts of the system interact dynamically.
6. **Describing Use Cases**: They are useful for illustrating the flow of control within a use case, showing how various components of the system interact during its execution.



3.3.4 Activity Diagram-1

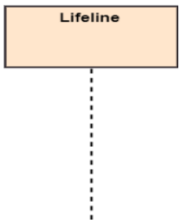
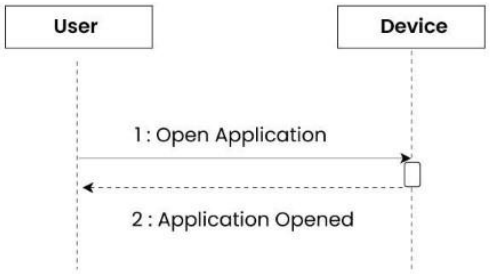
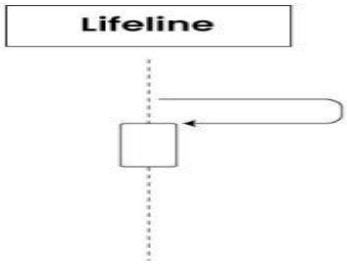


3.3.5 Activity Diagram-2

4. **SEQUENCE DIAGRAM:**

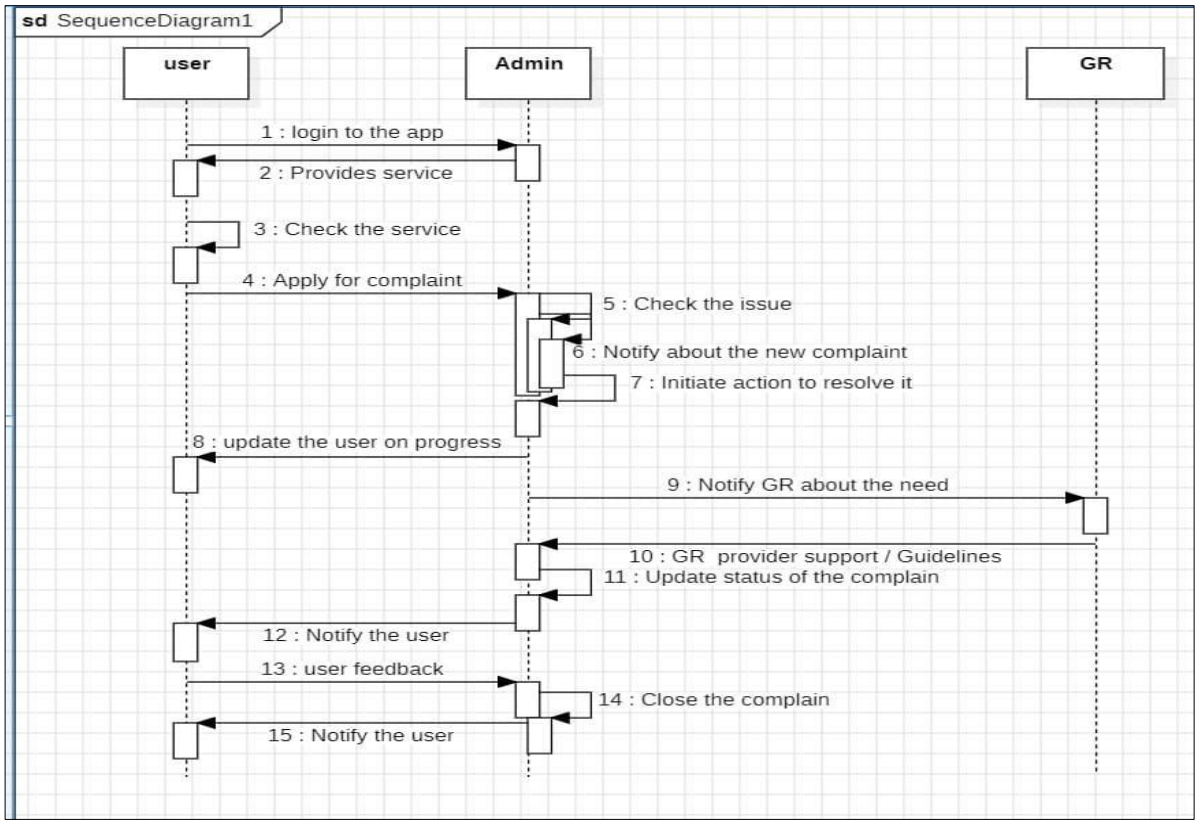
Sequence diagrams visually represent the interactions between objects or components in a system over time. They focus on the order and timing of messages or events exchanged between different system elements. The diagram captures how objects communicate with each other through a series of messages, providing a clear view of the sequence of operations or processes.

NOTATIONS OF SEQUENCE DIAGRAM:

Notation	Description
Lifeline 	An individual participant in the sequence diagram is represented by a lifeline. It is positioned at the top of the diagram.
Synchronous messages 	A synchronous message waits for a reply before the interaction can move forward.
Asynchronous Messages	An asynchronous message does not wait for a reply from the receiver.
Self Message 	Certain scenarios might arise where the object needs to send a message to itself

USE OF SEQUENCE DIAGRAM:

1. **Visualizing Dynamic Behavior**: Sequence diagrams depict how objects or systems interact with each other in a sequential manner, making it easier to understand dynamic processes and workflows.
2. **Clear Communication**: They provide an intuitive way to convey system behavior, helping teams understand complex interactions without diving into code.
3. **Use Case Analysis**: Sequence diagrams are useful for analyzing and representing use cases, making it clear how specific processes are executed within a system.
4. **Designing System Architecture**: They assist in defining how various components or services in a system communicate, which is essential for designing complex, distributed systems or service-oriented architectures.
5. **Documenting System Behavior**: Sequence diagrams provide an effective way to document how different parts of a system work together, which can be useful for both developers and maintenance teams.
6. **Debugging and Troubleshooting**: By modeling the sequence of interactions, they help identify potential bottlenecks, inefficiencies, or errors in system processes.



3.3.6 Sequence Diagram

CHAPTER 4

SYSTEM DESIGN

SYSTEM DESIGN

4.1 Basic Module

The modular approach allows each feature of the GaonSetu App to be developed, tested, and managed independently, promoting maintainability and scalability. Additionally, dividing the app into these modules ensures clear responsibility areas, making it easier for the development team to assign tasks and for the app to grow with additional features in the future.

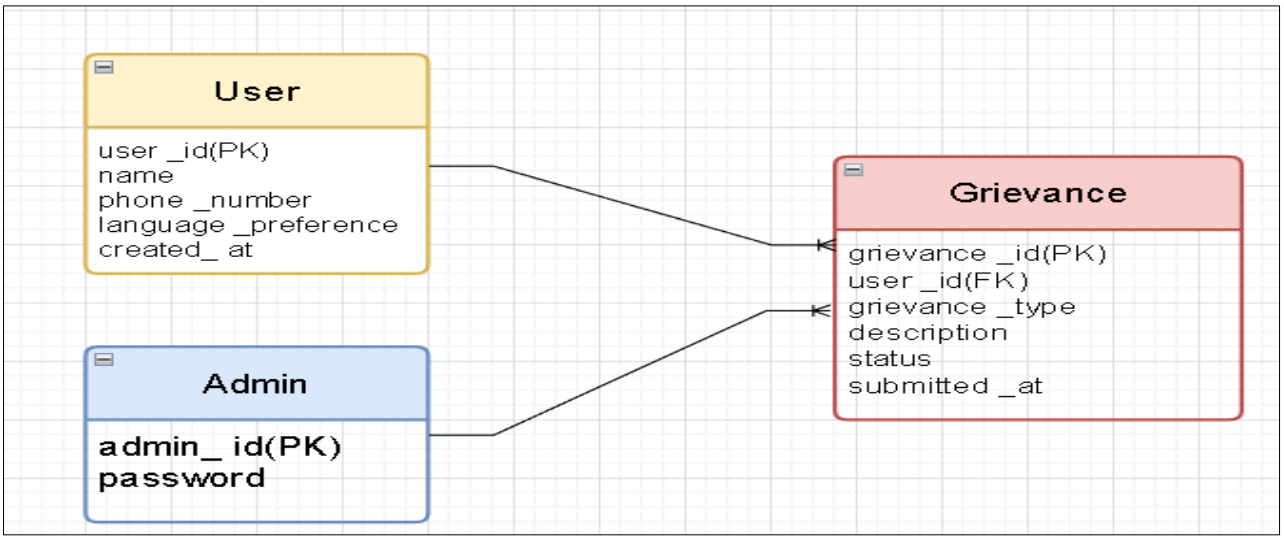
Here's a proposed Module Division for the GaonSetu App:

1. **User Management Module:**
   Handle all user-related actions, including registration, login, profile management, and authentication.
2. **Complaint Management Module:**
   Allow users to submit complaints, view their status, and track progress.
3. **Notification Module:**
   Keep users informed about updates on their complaints, donations, and important announcements.
4. **Donation Management Module:**
   Facilitate donations from users to fund specific village projects, track donation history, and integrate payment gateways.
5. **Government Rules and Assistance Module:**
   Provide villagers with access to relevant government rules, schemes, and assistance information.
6. **Admin and Government Dashboard Module**
   Provide tools for admins and government officials to manage complaints, view reports, and handle donations.
7. **Feedback and Rating Module:**
   Allow users to provide feedback on complaint resolution and rate the services.
8. **Payment and Transaction Module:**
   Handle all payment-related operations for donations and transactions.

4.2 Data Design

4.2.1 Schema Design

The schema design for the GaonSetu App is based on a relational database model to ensure efficient storage, retrieval, and management of data. The schema contains multiple entities such as User, Grievance and Admin. Relationships between these entities are defined to maintain data consistency and integrity.The goal of the schema design is to facilitate seamless interaction between different modules of the app, ensuring that all information is stored and retrieved efficiently.



4.2.2 Data Integrity and Constraints

1. **USER TABLE**

This table stores details of all users,including village,admins and government officials

Field Name	Data Types	Description
user_id	INT(Primary Key)	Unique identifier for each user.
name	VARCHAR(100)	Name of the user.
address	TEXT	Residential address of the user village_name

2. COMPLAINT TABLE

This table tracks complaints made by villagers.

Field Name	Data Types	Description
complain_id	INT(Primary Key)	Unique identifier for each complaint.
user_id	INT(Foreign Key)	Reference to the user who submitted the complaint.
category	ENUM('infrastructure', 'health','education','others')	Complaint category.
description	TEXT	Detailed description of the complaint.
submission_date	TIMESTAMP	Date and time when the complaint was submitted.
status	ENUM('submitted', 'in_progress','resolved')	Current status of the complaint.
resolution	TEXT	Description of how the complaint was resolved (if application).
assigned_to	INT(foreign key)	Reference to the government/admin user handling the complaint

3. DONATION TABLE

This table records information related to donation made by users/organization

Field Name	Data Types	Description
donation_id	INT(Primary Key)	Unique identifier for each donation
user_id	INT(Foreign Key)	Reference to the donar
amount	DECIMAL(10,2)	Amount donated
donation_date	TIMESTAMP	Date & time of the donation
project	VARCHAR(100)	The village project/cause for which the donation was made
payment_status	ENUM('pending','completed','failed')	Payment Status of the donation

4. NOTIFICATION TABLE

This table manages notification sent to the user regarding complaints,donation & updates.

Field Name	Data Type	Description
Notification_id	INT(Primary Key)	Unique identifier of each notification.
User_id	INT(Foreign Key)	Reference to the user receving the notification.
Message	TEXT	Notification message content.
Type	ENUM('complain_update', 'donation_acknowledgment', 'general')	Type of notification.
Send_date	TIMESTAMP	Date and time the notification was sent.
status	ENUM('read','unread')	Status indicating whether the notification was read.

5. GOVERNMENT RULES(GR) TABLE

This table contains information about government rules and schemes relevant to the users.

Field Name	Data Type	Description
gr_id	INT(Primary Key)	Unique identifier of each GR.
Title	VARCHAR(50)	Title of the GR.
Description	TEXT	Detailed description of the rule.
Start _date	DATE	Date when the rule was initiated.
end_date	DATE	End date of the rule.
eligibility	TEXT	Eligibility criteria for the rule.

4.3 Procedural Design

4.3.1 Logic Diagrams

The GaonSetu App follows a straightforward and intuitive workflow to empower villagers by providing grievance submission, scheme information, and simple admin controls. The logic is designed to ensure smooth navigation and secure data management.

1. Main Modules and Their Logic:

- User Registration and Login

  Step 1: User installs and launches the app.

  Step 2: Selects the preferred language (stored in SharedPreferences).

  Step 3: Enters mobile number to register/login.

  Step 4: OTP is generated and sent to the user's number.

  Step 5: On successful OTP verification:

  - If a new user, the profile is created in the database.
  - If an existing user, access to the main dashboard is granted.

Error Handling:

Invalid OTP → Re-prompt user.

OTP expiration → Request new OTP.

- Grievance Submission and Management

  Step 1: User submits grievance with relevant details (category, description, etc.).

  Step 2: Grievance request is stored in the database with a Pending status.

  Step 3: Admin reviews the grievance and updates the status (Approved/Rejected/In Progress).

  Step 4: User gets notified about the status change.

- Admin Controls

  Step 1: Admin logs in with credentials.

  Step 2: Admin views and manages grievances.

  Step 3: Updates the status of grievances as needed.

  Step 4: Admin can add or modify scheme information if required.

2.**Navigation Flow**:

Launcher Icon/Splash Screen → Language Selection → OTP Verification → Dashboard

User Module: Dashboard → Grievance Submission → Grievance Status → Scheme List

Admin Module: Dashboard → View Grievances → Update Status → Manage Schemes

4.3.2 Data Structures

1. User Data Management

Data Structure: HashMap / Dictionary

Purpose: Store and manage user details.

Attributes:

- user_id (Integer) – Unique identifier for each user.
- name (String) – User's name.
- language (String) – Preferred language of the user.
- phone (String) – Mobile number of the user.
- address (String) – Address of the user.

2. Grievance Management

Data Structure: List of HashMaps / Array of Dictionaries

Purpose: Manage grievance submissions and track status.

Attributes:

- grievance_id (Integer) – Unique identifier for each grievance.
- user_id (Integer) – User who submitted the grievance.
- category (String) – Type of grievance.
- description (String) – Details of the grievance.
- status (String) – Status of the grievance (Pending/Approved/Rejected).
- submitted_on (Date) – Submission date.

3. Admin Data Management

Data Structure: HashMap / Dictionary

Purpose: Store admin credentials and manage admin activities.

Attributes:

- username (String) – Admin username.
- password (String) – Admin password.

4. Language Preference Management

Data Structure: HashMap / Dictionary

Purpose: Store and manage user-selected language.

Attributes:

- user_id (Integer) – User identifier.
- selected_language (String) – Preferred language chosen by the user.

6. OTP Verification and Authentication

Data Structure: HashMap / Dictionary

Purpose: Store and verify OTP for login.

Attributes:

- phone (String) – Mobile number.
- otp (Integer) – One-time password sent to the user.
- timestamp (Date/Time) – OTP generation time.

4.3.3 Algorithm Design

1. User Registration and Login

This algorithm handles user registration and OTP-based login. It checks if the user is new or existing and updates the database accordingly.

Steps:

1. Start the process.
2. Prompt the user to enter their phone number.
3. Generate an OTP and send it to the user's phone.
4. Prompt the user to enter the received OTP.
5. If the entered OTP matches the generated OTP:
   a) Check if the user is a new or existing user.
   b) If the user is new:
   - Prompt the user to enter details like name, address, and preferred language.
   - Save the user details in the database.
   - Display a message "Registration successful."

   c) If the user is existing:
   - Display a message "Login successful."
6. If the entered OTP is incorrect:
   - Display a message "Invalid OTP. Try again."
7. Stop the process.

2. Grievance Submission

This algorithm allows users to submit grievances and track their status.

Steps:

1. Start the process.
2. Prompt the user to log in.
3. Display available grievance categories.
4. Prompt the user to select a category and enter grievance details.
5. Generate a unique grievance ID.
6. Save grievance details in the database with the status set to "Pending."
7. Display a message "Grievance submitted successfully."
8. Stop the process.

3. Language Preference Setup

This algorithm manages language selection and preference updates.

Steps:

1. Start the process.
2. Prompt the user to select a preferred language.
3. Save the selected language in the database.
4. Apply the selected language preference across the app.
5. Display a message "Language preference saved successfully."
6. Stop the process.

4. Admin Authentication

This algorithm verifies admin credentials for secure access.

Steps:

1. Start the process.
2. Prompt the admin to enter their username and password.
3. If the entered username and password are correct:
    - Display "Admin login successful."
4. If the credentials are incorrect:
    - Display "Invalid credentials. Access denied."
5. Stop the process.

4.4 User Interface Design

1. User Analysis

Target Users:

a. Primary Users: Villagers, rural residents, and local citizens.

b. Secondary Users:

- Government officials or local authorities (admin) managing grievances.
- App administrators who monitor and update data.

User Characteristics:

a. Limited technical knowledge.

b. Preference for a simple, easy-to-navigate interface.

c. Likely to use local languages for ease of understanding..

User Goals:

a. Submit grievances efficiently.

b. Check grievance status.

c. Access information about government schemes.

d. Receive updates and notifications.

2. Task Analysis

Core Tasks for Users:

- Login/Sign-Up:

- OTP-based authentication for secure login.

- Submit Grievances:

- Select category, enter description, and submit.

- Check Grievance Status:

- View the status of submitted grievances.

-Scheme Information:

- View available schemes and check eligibility.

- Language Selection:

- Choose a preferred language for app content.

Core Tasks for Admins:

-Login/Authenticate:

- Secure login for admin access.

-Manage Grievances:

- View, update, and change grievance status.

-Update Schemes and Content:

- Add or modify scheme information and app content.

3. Environment Analysis

Physical Environment: Users may access the app in villages with low network connectivity.

Technological Constraints: Many users may have basic Android smartphones with limited storage.

Accessibility Needs: The app should support large fonts, voice assistance, and multilingual options.

4. Mapping Requirements for UI Development

To meet user needs, the UI will follow these principles:

- **Simplicity:** Minimalist design similar to government apps.
- **Localization:** Support for multiple Indian languages.
- **Ease of Navigation:** Large buttons, clear labels, and intuitive icons.
- **Offline Support:** Ability to draft grievances offline and submit when online.
- **Dark Mode Support:** For better readability in low-light conditions.

5. External and Internal Components of UI

External Components (Visible to Users) :-

- Splash Screen: Animated logo with app name and a loading indicator.
- Home Screen: Main menu with options like Submit Grievance, View Schemes, Check Status.
- Forms: Simple forms with dropdowns, text fields, and voice input options.
- Buttons & Icons: Large, tappable elements with clear icons and labels.
- Language Selection: Leaf-shaped buttons (as per the village theme).

Internal Components (Backend UI Architecture) :-

- Navigation Controller: Manages screen transitions.
- State Management: Saves user preferences (e.g., selected language).
- API Integration: Fetches data for grievances, schemes, and OTP verification.
- Database UI Mapping: Retrieves stored grievances and user details.

6. Architecture of User Interface

The UI follows a layered architecture:

1. Presentation Layer: Displays UI elements (buttons, forms, lists).
2. Logic Layer: Handles user interactions (e.g., form validation, OTP authentication).
3. Data Layer: Fetches data from MySQL and displays it dynamically.

7. Rough Pictorial Views of UI

Home Screen

Components:

- Grievance Submission button

- View Government Schemes button
- Check Grievance Status button
- Settings & Language Selection

Grievance Submission Page

Components:

- Category Selection Dropdown
- Text/Voice Input for Complaint
- Attach Image (Optional)
- Submit Button

Admin Dashboard

Components:

- List of Pending Grievances
- Approve/Reject Buttons
- User Profile View

4.5 Security Issues

1.Real-Time Considerations and Security Issues:

The GaonSetu App aims to provide a seamless experience to rural users by allowing them to submit grievances, access scheme information, and track updates in real time. Since the app targets a user base with limited technical knowledge, it is essential to ensure that the interface is user-friendly and responsive. Real-time considerations involve ensuring smooth performance even under low bandwidth conditions, managing multiple user requests effectively, and providing timely grievance status updates. To achieve this, the app should leverage optimized APIs, background data synchronization, and push notifications to keep users informed without delays.

Security is a critical aspect of the GaonSetu App due to the sensitive nature of user data and grievance records. Key security challenges include preventing unauthorized access through OTP-based authentication and ensuring data privacy by encrypting sensitive information during storage and transmission. **Role-Based Access Control (RBAC)** restricts access to admin functionalities, while API rate limiting and secure communication protocols **(HTTPS)** protect against brute-force attacks and API misuse. Additionally, regular data backups and audit trails prevent data loss and enable traceability of actions, ensuring data integrity. To further enhance security, the app should incorporate input validation to prevent SQL injection attacks and conduct periodic vulnerability assessments to identify and mitigate potential threats. These real-time considerations and robust security measures ensure a reliable and secure platform for users and administrators alike.

2.Security Policy Plans and Architecture:

Security Plans

1. **Data Encryption:** Encrypt sensitive user data both during storage and transmission.
2. **Access Control:** Implement Role-Based Access Control (RBAC) to restrict unauthorized access.
3. **Authentication:** Use OTP-based login for secure user authentication.
4. **Regular Security Audits:** Conduct periodic checks to identify and fix vulnerabilities.
5. **Backup and Recovery:** Enable automatic database backups to prevent data loss.

Security Architecture

1. **Authentication Layer:** Handles OTP-based login and maintains secure sessions.
2. **API Security Layer:** Secures communication using HTTPS/TLS protocols.
3. **Access Control Layer:** Restricts admin and user access based on assigned roles.
4. **Audit and Monitoring Layer:** Maintains activity logs to detect suspicious behavior.

4.6 Test Case Design

Test Case ID	GS_TC_001	**Test Case Description**	Verify successful login with valid credentials.		
Created By	Prapti Singh & Sneha Yadav	**Reviewed By**		**Version**	

QA Tester's Log	Verified that user is able to log in successfully with correct credentials.

Tester's Name	Prapti Singh & Sneha Yadav	**Date Tested**	18-Jan-2025	**Test Case (Pass/Fail/Not Executed)**	Pass

S #	Prerequisites:
1	Internet connection (Wi-Fi or mobile data)
2	Android device or emulator with minimum version 7.0+
3	GaonSetu app installed and configured properly
4	Valid mobile number for OTP-based login
5	Test database with sample user accounts
6	MySQL database and backend APIs set upcand running
7	Admin panel configured for managing app content
8	Language selection configured in the app

CHAPTER 5

IMPLEMENTATION & TESTING

5.1 Implementation Approaches

1.Incremental Development Model :-

- The app is developed using the **Incremental Model**, where the core functionalities are implemented in small, manageable modules.
- Each module was tested and integrated sequentially to ensure smooth functionality and reduce errors.

2.Programming Language & Tools Used :-

- **Frontend:** Java (Android Studio) for app development.
- **Backend:** PHP for server-side scripting.
- **Database:** MySQL for storing and managing user and service data.
- **API Integration:** APIs are used to connect frontend and backend, ensuring secure data transmission.

3.Coding Standards & Guidelines :-

- **Naming Conventions:** Followed Java coding standards with camelCase for variables and PascalCase for classes.
- **Modular Coding:** Divided the application into smaller reusable modules to improve maintainability.
- **Error Handling:** Implemented try-catch blocks for error handling and ensured logging for debugging.

4.Version Control & Deployment :-

- Version Control: GitHub was used for version control and collaborative development.
- Deployment: App tested in both simulated and real environments before final deployment.

5.Security Standards :-

- Implemented secure APIs with proper encryption.
- Used OTP-based login for secure user verification.
- Ensured data encryption and secure session management.

6.Testing & Maintenance Plan :-

- **Unit Testing:** Tested individual components for expected functionality.
- **Integration Testing:** Verified seamless communication between modules.
- **User Acceptance Testing:** Ensured the app meets user requirements before deployment.

5.2 Coding Details And Code Efficiency

1. Splash Screen :-

- **SplashScreenActivity.java**

```
Package com.praptisingh.gaonsetuapplication;

import android.view.View;
import android.annotation.SuppressLint;
import android.content.Intent;
import android.os.Bundle;
import android.util.Log;
import android.os.Handler;
import android.view.animation.Animation;
import android.view.animation.AnimationUtils;
import android.widget.ImageView;
import androidx.appcompat.app.AppCompatActivity;

@SuppressLint("CustomSplashScreen")
public class SplashScreenActivity extends AppCompatActivity {

@Override
protected void onCreate(Bundle savedInstanceState) {
    super.onCreate(savedInstanceState);
    setContentView(R.layout.activity_splash_screen);

    // Logo Animation (Fade-In)
    ImageView logo = findViewById(R.id.gaonsetu_logo);
    Animation fadeIn = AnimationUtils.loadAnimation(this, R.anim.fade_in);
    logo.startAnimation(fadeIn);
    logo.setVisibility(View.VISIBLE);
    // Fade-in Animation for Logo
    fadeIn.setDuration(2000);
    // Circle Outline Animation (Rotation)
    ImageView circleOutline = findViewById(R.id.circle_outline);
```

```
        // Animation Listener to Start Circle Animation After Logo Animation Ends
        fadeIn.setAnimationListener(new Animation.AnimationListener() {
            @Override
            public void onAnimationStart(Animation animation) {
                logo.setVisibility(View.VISIBLE);
            }
            @Override
            public void onAnimationEnd(Animation animation) {
                // Start Circle Outline Animation after Logo Animation ends
                circleOutline.setVisibility(View.VISIBLE);
                Animation circleAnimation = AnimationUtils.loadAnimation(SplashScreenActivity.this,
R.anim.circle_animation);
                Log.d("AnimationCheck", "Circle Animation Started");
                circleOutline.startAnimation(circleAnimation);
                // Hold the splash screen for a while
                new Handler().postDelayed(() ->  {
                        // Start the next activity
                        Intent intent = new Intent(SplashScreenActivity.this,
LanguageSelectionActivity.class);
                        startActivity(intent);
                        finish();
                }, 5000); // Hold for 5 seconds
            }
            @Override
            public void onAnimationRepeat(Animation animation) {
                // Do nothing
            }
        });
        // Start Fade-in Animation
        logo.startAnimation(fadeIn);
    }
}
```

- **Activity_splash_screen.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_width="match_parent"
  android:layout_height="match_parent"
  android:background="@android:color/white">
<!-- Circular Outline -->
<ImageView
    android:id="@+id/circle_outline"
    android:layout_width="400dp"
    android:layout_height="400dp"
    android:layout_centerInParent="true"
    android:visibility="visible"
    android:src="@drawable/circle_outline"
    android:contentDescription="@string/circle_effect_desc" />
<!-- Logo -->
<ImageView
    android:id="@+id/gaonsetu_logo"
    android:layout_width="300dp"
    android:layout_height="310dp"
    android:layout_centerInParent="true"
    android:src="@drawable/gaonsetu_logo"
    android:visibility="visible"
    android:contentDescription="@string/gaonsetu_logo_desc" />
</RelativeLayout>
```

2. Language Screen

- **LanguageSelectionActivity.java**

```
package com.praptisingh.gaonsetuapplication;
import android.content.Intent;
import android.content.SharedPreferences;
import android.os.Bundle;
import android.widget.AdapterView;
import android.widget.ArrayAdapter;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import java.util.Locale;
public class LanguageSelectionActivity extends AppCompatActivity {
SharedPreferences prefs;
private final String[] languages = {"English", "हिन्दी", "বাংলা", "ગુજરાતી", "मराठी", "தமிழ்",
"తెలుగు", "ಕನ್ನಡ", "മലയാളം", "ଓଡ଼ିଆ", "ਪੰਜਾਬੀ", "اردو"};
@Override
protected void onCreate(Bundle savedInstanceState) {
super.onCreate(savedInstanceState);
setContentView(R.layout.activity_language_selection);
recyclerView = findViewById(R.id.recyclerViewLanguages);
recyclerView.setLayoutManager(new LinearLayoutManager(this));
recyclerView.setAdapter(new LanguageAdapter(languages));
ArrayAdapter<String> adapter = new ArrayAdapter<>(this, android.R.layout.simple_list_item_1,
languages);
lvLanguages.setAdapter(adapter);
lvLanguages.setOnItemClickListener((AdapterView<?> parent, android.view.View view, int
position, long id) -> {
String selectedLanguage = languages[position];
setAppLanguage(selectedLanguage);
// Save the selected language
SharedPreferences sharedPreferences = getSharedPreferences("LanguagePref",
MODE_PRIVATE);
SharedPreferences.Editor editor = sharedPreferences.edit();
```

```
            editor.putString("SelectedLanguage", selectedLanguage);
            editor.apply();
            Toast.makeText(LanguageSelectionActivity.this, "Language set to: " + selectedLanguage,
Toast.LENGTH_SHORT).show();
            // Restart the activity to apply the language
            Intent intent = new Intent(LanguageSelectionActivity.this, MainActivity.class);
            startActivity(intent);
            finish();
        });
    }
    private void setAppLanguage(String language) {
        Locale locale;
        switch (language) {
            case "हिन्दी":
                locale = new Locale("hi");
                break;
            case "বাংলা":
                locale = new Locale("bn");
                break;
            case "ગુજરાતી":
                locale = new Locale("gu");
                break;
            case "मराठी":
                locale = new Locale("mr");
                break;
            case "தமிழ்":
                locale = new Locale("ta");
                break;
            case "తెలుగు":
                locale = new Locale("te");
                break;
            case "ಕನ್ನಡ":
                locale = new Locale("kn");
```

```
            break;
        case "മലയാളം":
            locale = new Locale("ml");
            break;
        case "ଓଡ଼ିଆ":
            locale = new Locale("or");
            break;
        case "ਪੰਜਾਬੀ":
            locale = new Locale("pa");
            break;
        case "اردو":
            locale = new Locale("ur");
            break;
        default:
            locale = new Locale("en");
            break;
    }
    Locale.setDefault(locale);
    android.content.res.Configuration config = new android.content.res.Configuration();
    config.setLocale(locale);
    getResources().updateConfiguration(config, getResources().getDisplayMetrics());
  }
}
```

- **Activity_language_selection.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.constraintlayout.widget.ConstraintLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
   xmlns:app="http://schemas.android.com/apk/res-auto"
   xmlns:tools="http://schemas.android.com/tools"
   android:layout_width="match_parent"
   android:layout_height="match_parent"
   android:background="@android:color/white"
   tools:context=".MainActivity">
   <!-- Curved Header -->
   <View
      android:id="@+id/curvedHeader"
      android:layout_width="412dp"
      android:layout_height="140dp"
      android:background="@drawable/s_curve_header"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintHorizontal_bias="1.0"
      app:layout_constraintStart_toStartOf="parent"
      app:layout_constraintTop_toTopOf="parent" />

   <TextView
      android:id="@+id/textSelectLanguage"
      android:layout_width="wrap_content"
      android:layout_height="wrap_content"
      android:gravity="center"
      android:text="@string/select_language"
      android:textColor="@android:color/white"
      android:textSize="22sp"
      android:textStyle="bold"
      app:layout_constraintBottom_toBottomOf="@id/curvedHeader"
      app:layout_constraintEnd_toEndOf="parent"
      app:layout_constraintStart_toStartOf="parent"
```

```
    app:layout_constraintTop_toTopOf="@id/curvedHeader"
    tools:layout_editor_absoluteX="132dp"
    tools:layout_editor_absoluteY="70dp" />
 <!-- Grid Layout for Leaf Buttons -->
<GridLayout
   android:id="@+id/languageGrid"
   android:layout_width="match_parent"
   android:layout_height="wrap_content"
   android:columnCount="3"
   android:padding="16dp"
   android:rowCount="3"
   app:layout_constraintBottom_toBottomOf="parent"
   app:layout_constraintEnd_toEndOf="parent"
   app:layout_constraintHorizontal_bias="0.0"
   app:layout_constraintStart_toStartOf="parent"
   app:layout_constraintTop_toBottomOf="@+id/curvedHeader"
   app:layout_constraintVertical_bias="0.0">
   <androidx.constraintlayout.widget.ConstraintLayout
      android:layout_width="wrap_content"
      android:layout_height="wrap_content">
      <ImageView
         android:id="@+id/leafHindi"
         android:layout_width="120dp"
         android:layout_height="140dp"
         android:scaleType="fitCenter"
         android:src="@drawable/leaf_image"
         app:layout_constraintStart_toStartOf="parent"
         app:layout_constraintTop_toTopOf="parent"/>
      <!-- Language Text (Centered Inside Leaf) -->
      <TextView
         android:id="@+id/textHindi"
         android:layout_width="wrap_content"
         android:layout_height="wrap_content"
         android:text="@string/Hindi"
```

```
        android:textSize="20sp"
        android:textColor="#000000"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafHindi"
        app:layout_constraintBottom_toBottomOf="@id/leafHindi"
        app:layout_constraintStart_toStartOf="@id/leafHindi"
        app:layout_constraintEnd_toEndOf="@id/leafHindi"/>
    <!-- RadioButton (Perfectly Positioned at the Tip of Leaf) -->
    <RadioButton
        android:id="@+id/radioHindi"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintEnd_toEndOf="@id/leafHindi"
        app:layout_constraintTop_toTopOf="@id/leafHindi"
        android:minWidth="50dp"
        android:minHeight="52dp"
        android:layout_marginEnd="-8dp"
        android:layout_marginTop="17dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <!-- Bengali -->
    <ImageView
        android:id="@+id/leafBengali"
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toEndOf="@id/leafHindi"
        app:layout_constraintTop_toTopOf="@id/leafHindi"
        android:layout_marginStart="16dp"
        android:layout_marginTop="8dp"
        android:background="@android:color"/>
```

```
    <TextView
        android:id="@+id/textBengali"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Bengali"
        android:textColor="#000000"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafBengali"
        app:layout_constraintBottom_toBottomOf="@id/leafBengali"
        app:layout_constraintStart_toStartOf="@id/leafBengali"
        app:layout_constraintEnd_toEndOf="@id/leafBengali"/>
    <RadioButton
        android:id="@+id/radioBengali"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafBengali"
        app:layout_constraintTop_toTopOf="@id/leafBengali"
        app:layout_constraintEnd_toEndOf="@id/leafBengali"
        android:minWidth="53dp"
        android:minHeight="52dp"
        android:layout_marginStart="88dp"
        android:layout_marginTop="17dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <!-- Marathi -->
    <ImageView
        android:id="@+id/leafMarathi"
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
```

```
        app:layout_constraintStart_toEndOf="@id/leafBengali"
        app:layout_constraintTop_toTopOf="@id/leafBengali"
        android:layout_marginStart="20dp" />
    <TextView
        android:id="@+id/textMarathi"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Marathi"
        android:textColor="#000000"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafMarathi"
        app:layout_constraintBottom_toBottomOf="@id/leafMarathi"
        app:layout_constraintStart_toStartOf="@id/leafMarathi"
        app:layout_constraintEnd_toEndOf="@id/leafMarathi"/>
    <RadioButton
        android:id="@+id/radioMarathi"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafMarathi"
        app:layout_constraintTop_toTopOf="@id/leafMarathi"
        app:layout_constraintEnd_toEndOf="@id/leafMarathi"
        android:minWidth="59dp"
        android:minHeight="55dp"
        android:layout_marginStart="90dp"
        android:layout_marginTop="17dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/leafGujarati"
        android:layout_width="120dp"
        android:layout_height="140dp"
```

```
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <TextView
        android:id="@+id/textGujarati"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Gujarati"
        android:textSize="20sp"
        android:textColor="#000000"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafGujarati"
        app:layout_constraintBottom_toBottomOf="@id/leafGujarati"
        app:layout_constraintStart_toStartOf="@id/leafGujarati"
        app:layout_constraintEnd_toEndOf="@id/leafGujarati"/>
    <RadioButton
        android:id="@+id/radioGujarati"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintEnd_toEndOf="@id/leafGujarati"
        app:layout_constraintTop_toTopOf="@id/leafGujarati"
        android:minWidth="55dp"
        android:minHeight="52dp"
        android:layout_marginEnd="-11dp"
        android:layout_marginTop="17dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/leafTamil"
        android:layout_width="120dp"
        android:layout_height="140dp"
```

```
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toEndOf="@id/leafGujarati"
        app:layout_constraintTop_toTopOf="@id/leafGujarati"
        android:layout_marginStart="20dp"/>
    <TextView
        android:id="@+id/textTamil"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Tamil"
        android:textColor="#000000"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafTamil"
        app:layout_constraintBottom_toBottomOf="@id/leafTamil"
        app:layout_constraintStart_toStartOf="@id/leafTamil"
        app:layout_constraintEnd_toEndOf="@id/leafTamil"/>
    <RadioButton
        android:id="@+id/radioTamil"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafTamil"
        app:layout_constraintTop_toTopOf="@id/leafTamil"
        app:layout_constraintEnd_toEndOf="@id/leafTamil"
        android:minWidth="59dp"
        android:minHeight="59dp"
        android:layout_marginStart="90dp"
        android:layout_marginTop="15dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/leafTelugu"
```

```
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toEndOf="@id/leafTamil"
        app:layout_constraintTop_toTopOf="@id/leafTamil"
        android:layout_marginStart="20dp" />
    <TextView
        android:id="@+id/textTelugu"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Telugu"
        android:textColor="#000000"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafTelugu"
        app:layout_constraintBottom_toBottomOf="@id/leafTelugu"
        app:layout_constraintStart_toStartOf="@id/leafTelugu"
        app:layout_constraintEnd_toEndOf="@id/leafTelugu"/>
    <RadioButton
        android:id="@+id/radioTelugu"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafTelugu"
        app:layout_constraintTop_toTopOf="@id/leafTelugu"
        app:layout_constraintEnd_toEndOf="@id/leafTelugu"
        android:minWidth="59dp"
        android:minHeight="59dp"
        android:layout_marginStart="90dp"
        android:layout_marginTop="15dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
```

```
    <ImageView
        android:id="@+id/leafKannada"
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toStartOf="parent"
        app:layout_constraintTop_toTopOf="parent" />
    <TextView
        android:id="@+id/textKannada"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Kannada"
        android:textSize="20sp"
        android:textColor="#000000"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafKannada"
        app:layout_constraintBottom_toBottomOf="@id/leafKannada"
        app:layout_constraintStart_toStartOf="@id/leafKannada"
        app:layout_constraintEnd_toEndOf="@id/leafKannada"/>
    <RadioButton
        android:id="@+id/radioKannada"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintEnd_toEndOf="@id/leafKannada"
        app:layout_constraintTop_toTopOf="@id/leafKannada"
        android:minWidth="59dp"
        android:minHeight="59dp"
        android:layout_marginEnd="-15dp"
        android:layout_marginTop="15dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
```

```
    <ImageView
        android:id="@+id/leafMalayalam"
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"
        app:layout_constraintStart_toEndOf="@id/leafKannada"
        app:layout_constraintTop_toTopOf="@id/leafKannada"
        android:layout_marginStart="20dp"/>
    <TextView
        android:id="@+id/textMalayalam"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Malayalam"
        android:textColor="#000000"
        android:textSize="15sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafMalayalam"
        app:layout_constraintBottom_toBottomOf="@id/leafMalayalam"
        app:layout_constraintStart_toStartOf="@id/leafMalayalam"
        app:layout_constraintEnd_toEndOf="@id/leafMalayalam"/>
    <RadioButton
        android:id="@+id/radioMalayalam"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafMalayalam"
        app:layout_constraintTop_toTopOf="@id/leafMalayalam"
        app:layout_constraintEnd_toEndOf="@id/leafMalayalam"
        android:minWidth="59dp"
        android:minHeight="59dp"
        android:layout_marginStart="90dp"
        android:layout_marginTop="15dp"/>
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
```

```
android:layout_width="wrap_content"
android:layout_height="wrap_content">
<ImageView
    android:id="@+id/leafOdia"
    android:layout_width="120dp"
    android:layout_height="140dp"
    android:scaleType="fitCenter"
    android:src="@drawable/leaf_image"
    app:layout_constraintStart_toEndOf="@id/leafMalayalam"
    app:layout_constraintTop_toTopOf="@id/leafMalayalam"
    android:layout_marginStart="20dp" />
<TextView
    android:id="@+id/textOdia"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="@string/Odia"
    android:textColor="#000000"
    android:textSize="20sp"
    android:textStyle="bold"
    app:layout_constraintTop_toTopOf="@id/leafOdia"
    app:layout_constraintBottom_toBottomOf="@id/leafOdia"
    app:layout_constraintStart_toStartOf="@id/leafOdia"
    app:layout_constraintEnd_toEndOf="@id/leafOdia"/>
<RadioButton
    android:id="@+id/radioOdia"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    app:layout_constraintStart_toStartOf="@id/leafOdia"
    app:layout_constraintTop_toTopOf="@id/leafOdia"
    app:layout_constraintEnd_toEndOf="@id/leafOdia"
    android:minWidth="59dp"
    android:minHeight="59dp"
    android:layout_marginStart="90dp"
    android:layout_marginTop="15dp"/>
```

```
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
</androidx.constraintlayout.widget.ConstraintLayout>
<androidx.constraintlayout.widget.ConstraintLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content">
    <ImageView
        android:id="@+id/leafPunjabi"
        android:layout_width="120dp"
        android:layout_height="140dp"
        android:scaleType="fitCenter"
        android:src="@drawable/leaf_image"/>
    <TextView
        android:id="@+id/textPunjabi"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/Punjabi"
        android:textColor="#000000"
        android:textSize="20sp"
        android:textStyle="bold"
        app:layout_constraintTop_toTopOf="@id/leafPunjabi"
        app:layout_constraintBottom_toBottomOf="@id/leafPunjabi"
        app:layout_constraintStart_toStartOf="@id/leafPunjabi"
        app:layout_constraintEnd_toEndOf="@id/leafPunjabi"/>
    <RadioButton
        android:id="@+id/radioPunjabi"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        app:layout_constraintStart_toStartOf="@id/leafPunjabi"
        app:layout_constraintTop_toTopOf="@id/leafPunjabi"
        app:layout_constraintEnd_toEndOf="@id/leafPunjabi"
        android:minWidth="59dp"
```

```
            android:minHeight="59dp"
            android:layout_marginStart="90dp"
            android:layout_marginTop="15dp"/>
        </androidx.constraintlayout.widget.ConstraintLayout>
    </GridLayout>
</androidx.constraintlayout.widget.ConstraintLayout>
```

3.Login Screen

- **LoginActivity.java**

```
package com.example.gaonsetu;
import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class LoginActivity extends AppCompatActivity {

    private EditText phoneNumber, otpField;
    private Button sendOtpButton, verifyOtpButton;
    private String generatedOtp = "1234";  // Dummy OTP for testing

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_login);

        // Initialize UI Components
        phoneNumber = findViewById(R.id.phoneNumber);
        otpField = findViewById(R.id.otpField);
        sendOtpButton = findViewById(R.id.sendOtpButton);
```

```
        verifyOtpButton = findViewById(R.id.verifyOtpButton);

        // Send OTP Button Click Listener
        sendOtpButton.setOnClickListener(v -> {
            String phone = phoneNumber.getText().toString().trim();
            if (phone.isEmpty() || phone.length() != 10) {
                Toast.makeText(LoginActivity.this, "Enter a valid phone number!",
Toast.LENGTH_SHORT).show();
            } else {
                sendOtp(phone);
            }
        });

        // Verify OTP Button Click Listener
        verifyOtpButton.setOnClickListener(v -> {
            String enteredOtp = otpField.getText().toString().trim();
            if (enteredOtp.equals(generatedOtp)) {
                Toast.makeText(LoginActivity.this, "Login Successful!", Toast.LENGTH_SHORT).show();
                // Redirect to Home Screen
                Intent intent = new Intent(LoginActivity.this, HomeActivity.class);
                startActivity(intent);
                finish();
            } else {
                Toast.makeText(LoginActivity.this, "Invalid OTP. Try again!",
Toast.LENGTH_SHORT).show();
            }
        });
    }

    // Function to Send OTP (Dummy for Testing)
    private void sendOtp(String phone) {
        // Logic for sending OTP via API (to be added later)
        Toast.makeText(this, "OTP Sent to " + phone, Toast.LENGTH_SHORT).show();
    }
```

}

- **activity_login.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="20dp">

    <!-- Phone Number Input -->
    <EditText
        android:id="@+id/phoneNumber"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:hint="Enter Phone Number"
        android:inputType="phone"
        android:padding="12dp"
        android:background="@android:drawable/editbox_background_normal" />

    <!-- Send OTP Button -->
    <Button
        android:id="@+id/sendOtpButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Send OTP"
        android:layout_below="@id/phoneNumber"
        android:layout_marginTop="10dp"
        android:padding="12dp" />

    <!-- OTP Input Field -->
    <EditText
        android:id="@+id/otpField"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
```

```
        android:hint="Enter OTP"
        android:inputType="number"
        android:padding="12dp"
        android:layout_below="@id/sendOtpButton"
        android:layout_marginTop="10dp" />

    <!-- Verify OTP Button -->
    <Button
        android:id="@+id/verifyOtpButton"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Verify OTP"
        android:layout_below="@id/otpField"
        android:layout_marginTop="10dp"
        android:padding="12dp" />
</RelativeLayout>
```

4. Main Screen:-

- **<u>MainActivity.java</u>**

```
package com.example.gaonsetu;
import android.os.Bundle;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
import com.google.android.material.bottomnavigation.BottomNavigationView;
import android.view.MenuItem;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        // Bottom Navigation Initialization
        BottomNavigationView bottomNavigationView = findViewById(R.id.bottom_navigation);
```

```
        // Set Default Fragment (Home)
        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container,
                    new HomeFragment()).commit();
        }
        // Navigation Item Selected Listener
        bottomNavigationView.setOnItemSelectedListener(item -> {
            Fragment selectedFragment = null;

            // Switch case for navigation items
            int itemId = item.getItemId();
            if (itemId == R.id.nav_home) {
                selectedFragment = new HomeFragment();
            } else if (itemId == R.id.nav_announcement) {
                selectedFragment = new AnnouncementFragment();
            } else if (itemId == R.id.nav_complain) {
                selectedFragment = new ComplainFragment();
            } else if (itemId == R.id.nav_profile) {
                selectedFragment = new ProfileFragment();
            }
            // Load Selected Fragment
            if (selectedFragment != null) {
                getSupportFragmentManager().beginTransaction().replace(R.id.fragment_container,
                        selectedFragment).commit();
            }
            return true;
        });
    }
}
```

- **activity_main.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
```

```
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <!-- Fragment Container for Switching Screens -->
    <FrameLayout
        android:id="@+id/fragment_container"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_above="@id/bottom_navigation" />
    <!-- Bottom Navigation Menu -->
    <com.google.android.material.bottomnavigation.BottomNavigationView
        android:id="@+id/bottom_navigation"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:layout_alignParentBottom="true"
        app:itemIconTint="@color/nav_item_color"
        app:itemTextColor="@color/nav_item_color"
        app:menu="@menu/bottom_navigation_menu" />
</RelativeLayout>
```

- **bottom_navigation_menu.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item
        android:id="@+id/nav_home"
        android:icon="@drawable/ic_home"
        android:title="Home" />
    <item
        android:id="@+id/nav_announcement"
        android:icon="@drawable/ic_announcement"
        android:title="Announcement" />
    <item
        android:id="@+id/nav_complain"
        android:icon="@drawable/ic_complain"
        android:title="Complain" />
    <item
```

```
        android:id="@+id/nav_profile"
        android:icon="@drawable/ic_profile"
        android:title="Profile" />
</menu>
```

- **HomeFragment.java**

```
package com.example.gaonsetu;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

public class HomeFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        return inflater.inflate(R.layout.fragment_home, container, false);
    }
}
```

5.Complaint Form

- **ComplaintFormActivity.java**

```
package com.example.gaonsetu;

import android.os.Bundle;
import android.text.TextUtils;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Spinner;
```

```
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class ComplaintFormActivity extends AppCompatActivity {

    private EditText etFullName, etEmail, etPhoneNumber, etLocation, etDescription;
    private Spinner spinnerCategory;
    private Button btnSubmitComplaint;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_complaint_form);

        // Initialize Views
        etFullName = findViewById(R.id.et_full_name);
        etEmail = findViewById(R.id.et_email);
        etPhoneNumber = findViewById(R.id.et_phone_number);
        etLocation = findViewById(R.id.et_location);
        etDescription = findViewById(R.id.et_description);
        spinnerCategory = findViewById(R.id.spinner_category);
        btnSubmitComplaint = findViewById(R.id.btn_submit_complaint);

        // Submit Button Click Listener
        btnSubmitComplaint.setOnClickListener(v -> submitComplaint());
    }

    private void submitComplaint() {
        // Get Input Values
        String fullName = etFullName.getText().toString().trim();
        String email = etEmail.getText().toString().trim();
        String phoneNumber = etPhoneNumber.getText().toString().trim();
        String location = etLocation.getText().toString().trim();
        String category = spinnerCategory.getSelectedItem().toString();
```

```
        String description = etDescription.getText().toString().trim();

        // Validate Inputs
        if (TextUtils.isEmpty(fullName) || TextUtils.isEmpty(email) ||
                TextUtils.isEmpty(phoneNumber) || TextUtils.isEmpty(location) ||
                TextUtils.isEmpty(description)) {
            Toast.makeText(this, "Please fill all the fields", Toast.LENGTH_SHORT).show();
            return;
        }
        // Send Complaint Data to Backend (Dummy for Now)
        Toast.makeText(this, "Complaint Submitted Successfully!", Toast.LENGTH_SHORT).show();
        // Clear Form
        clearForm();
    }
    private void clearForm() {
        etFullName.setText("");
        etEmail.setText("");
        etPhoneNumber.setText("");
        etLocation.setText("");
        etDescription.setText("");
        spinnerCategory.setSelection(0);
    }
}
```

- **activity_complaint_form.xml**

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp"
    android:background="#F5F5F5">

    <LinearLayout
        android:layout_width="match_parent"
```

```
android:layout_height="wrap_content"
android:orientation="vertical">

<!-- Header -->
<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="Submit a Complaint"
    android:textSize="18sp"
    android:textColor="#FFFFFF"
    android:gravity="center"
    android:padding="12dp"
    android:background="#9C27B0" />

<TextView
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:text="We're here to help. Please provide details about your issue."
    android:textColor="#757575"
    android:gravity="center"
    android:padding="8dp" />

<!-- Complaint Details Section -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Complaint Details"
    android:textStyle="bold"
    android:textSize="16sp"
    android:paddingTop="12dp"
    android:textColor="#4A4A4A" />

<!-- Input Fields -->
<EditText
```

```
    android:id="@+id/et_full_name"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Full Name"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg"
    android:textColorHint="#9E9E9E" />

<EditText
    android:id="@+id/et_email"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Email Address"
    android:inputType="textEmailAddress"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg" />

<EditText
    android:id="@+id/et_phone_number"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Phone Number"
    android:inputType="phone"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg" />

<EditText
    android:id="@+id/et_location"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Location"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg" />
```

```
<!-- Complaint Category Dropdown -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Complaint Category"
    android:textColor="#4A4A4A"
    android:paddingTop="8dp" />
<Spinner
    android:id="@+id/spinner_category"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:entries="@array/complaint_categories"
    android:padding="8dp" />
<!-- Complaint Description -->
<TextView
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:text="Complaint Description"
    android:textStyle="bold"
    android:textSize="16sp"
    android:paddingTop="12dp"
    android:textColor="#4A4A4A" />
<EditText
    android:id="@+id/et_description"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    android:hint="Briefly describe your issue"
    android:gravity="top|start"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg"
    android:textColorHint="#9E9E9E" />
<!-- Submit Button -->
<Button
    android:id="@+id/btn_submit_complaint"
```

```
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Submit Complaint"
        android:background="@drawable/button_bg"
        android:textColor="@android:color/white"
        android:textSize="16sp"
        android:layout_marginTop="12dp" />
    </LinearLayout>
</ScrollView>
```

Profile Page :

Profile.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:orientation="vertical"
    android:padding="20dp"
    android:gravity="center"
    android:background="@color/white">

    <TextView
        android:id="@+id/tvUsername"
        android:layout_width="match_parent"
        android:layout_height="match_parent"
        android:layout_marginTop="20dp"
        android:text="Welcome, User"
        android:textColor="@color/black"
        android:textSize="18sp" />
</LinearLayout>
```

Profile.java

```
package com.example.gaon;
import android.content.SharedPreferences;
```

```
import android.os.Bundle;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class profile extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_profile);
        // Find TextView by ID
        TextView tvUsername = findViewById(R.id.tvUsername);
        // Retrieve username from SharedPreferences
        SharedPreferences sharedPreferences = getSharedPreferences("user_session",
MODE_PRIVATE);
        String username = sharedPreferences.getString("username", "").trim();
        // If username is empty, set to "Guest"
        if (username.isEmpty()) {
            username = "Guest";
        }

        // Set username in TextView
        tvUsername.setText(username);
        // Show a Toast message
        Toast.makeText(this, "Logged in as: " + username, Toast.LENGTH_SHORT).show();
    }
}
```

- **Admin**

activity_admin_login.xml

```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    xmlns:app="http://schemas.android.com/apk/res-auto"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
```

```
android:padding="24dp">

<!-- Admin Logo -->
<ImageView
    android:id="@+id/iv_admin_logo"
    android:layout_width="120dp"
    android:layout_height="120dp"
    android:layout_centerHorizontal="true"
    android:src="@drawable/ic_admin_logo"
    android:layout_marginTop="40dp" />

<!-- Username Input -->
<EditText
    android:id="@+id/et_username"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Username"
    android:layout_below="@id/iv_admin_logo"
    android:layout_marginTop="24dp"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg"
    android:textSize="16sp" />

<!-- Password Input -->
<EditText
    android:id="@+id/et_password"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:hint="Password"
    android:inputType="textPassword"
    android:layout_below="@id/et_username"
    android:layout_marginTop="12dp"
    android:padding="12dp"
    android:background="@drawable/edit_text_bg"
```

```
        android:textSize="16sp" />

    <!-- Login Button -->
    <Button
        android:id="@+id/btn_login"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:text="Login"
        android:layout_below="@id/et_password"
        android:layout_marginTop="20dp"
        android:background="@drawable/button_bg"
        android:textColor="@android:color/white"
        android:textSize="16sp" />
</RelativeLayout>
```

AdminLoginActivity.java

```
package com.example.gaonsetuadmin;

import android.content.Intent;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class AdminLoginActivity extends AppCompatActivity {

    // Static credentials for admin login
    private final String ADMIN_USERNAME = "gaonsetu";
    private final String ADMIN_PASSWORD = "admin123";
```

```
    private EditText etUsername, etPassword;
    private Button btnLogin;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_admin_login);

        // Initialize views
        etUsername = findViewById(R.id.et_username);
        etPassword = findViewById(R.id.et_password);
        btnLogin = findViewById(R.id.btn_login);

        // Button Click Listener
        btnLogin.setOnClickListener(v -> {
            String username = etUsername.getText().toString().trim();
            String password = etPassword.getText().toString().trim();

            // Validate Admin Credentials
            if (username.equals(ADMIN_USERNAME) && password.equals(ADMIN_PASSWORD)) {
                Toast.makeText(AdminLoginActivity.this, "Login Successful!",
Toast.LENGTH_SHORT).show();
                // Navigate to Admin Dashboard
                Intent intent = new Intent(AdminLoginActivity.this, AdminMainActivity.class);
                startActivity(intent);
                finish();
            } else {
                Toast.makeText(AdminLoginActivity.this, "Invalid Username or Password!",
Toast.LENGTH_SHORT).show();
            }
        });
    }
}
```

```
CREATE DATABASE
 gaonsetu_db;

USE gaonsetu_db;

-- Users Table
CREATE TABLE users (
   id INT PRIMARY KEY AUTO_INCREMENT,
   name VARCHAR(100),
   phone VARCHAR(15) UNIQUE,
   otp VARCHAR(6),
   language_selected VARCHAR(50),
   created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Admin Table
CREATE TABLE admin (
   id INT PRIMARY KEY AUTO_INCREMENT,
   username VARCHAR(50),
   password VARCHAR(255)
);

-- Language Preferences Table
CREATE TABLE language_preferences (
   id INT PRIMARY KEY AUTO_INCREMENT,
   user_id INT,
   language_selected VARCHAR(50),
   FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
CREATE TABLE ServiceListings (
   service_id INT PRIMARY KEY AUTO_INCREMENT,
   user_id INT,
   service_title VARCHAR(100),
```

```
    service_description TEXT,
    category VARCHAR(50),

    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);
CREATE TABLE Transactions (
    transaction_id INT PRIMARY KEY AUTO_INCREMENT,
    user_id INT,
    amount DECIMAL(10, 2),
    payment_status ENUM('success', 'failed', 'pending') DEFAULT 'pending',
    payment_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (user_id) REFERENCES Users(user_id) ON DELETE CASCADE
);



-- Create the database
CREATE DATABASE gaonsetu_db;

-- Use the database
USE gaonsetu_db;

-- Create admin_users table
CREATE TABLE admin_users (
    admin_id INT PRIMARY KEY AUTO_INCREMENT,
    username VARCHAR(50) UNIQUE NOT NULL,
    password VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    role ENUM('SuperAdmin', 'Moderator') DEFAULT 'Moderator',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create villagers table
CREATE TABLE villagers (
```

```
    villager_id INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100) NOT NULL,
    phone_number VARCHAR(15) UNIQUE NOT NULL,
    address TEXT NULL,
    language VARCHAR(20) DEFAULT 'Hindi',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create services table
CREATE TABLE services (
    service_id INT PRIMARY KEY AUTO_INCREMENT,
    service_name VARCHAR(100) NOT NULL,
    description TEXT NULL,
    status ENUM('Active', 'Inactive') DEFAULT 'Active',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create requests table
CREATE TABLE requests (
    request_id INT PRIMARY KEY AUTO_INCREMENT,
    villager_id INT,
    service_id INT,
    status ENUM('Pending', 'Approved', 'Rejected', 'Completed') DEFAULT 'Pending',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    FOREIGN KEY (villager_id) REFERENCES villagers(villager_id) ON DELETE CASCADE,
    FOREIGN KEY (service_id) REFERENCES services(service_id) ON DELETE CASCADE
);

-- Create notifications table
CREATE TABLE notifications (
    notification_id INT PRIMARY KEY AUTO_INCREMENT,
    message TEXT NOT NULL,
    target_audience ENUM('All', 'Admins', 'Villagers') DEFAULT 'All',
    created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
```

```
);

-- Sample Data for admin_users
INSERT INTO admin_users (username, password, email, role)
VALUES
('admin', 'admin123', 'admin@gaonsetu.com', 'SuperAdmin'),
('moderator', 'mod123', 'mod@gaonsetu.com', 'Moderator');

-- Sample Data for services
INSERT INTO services (service_name, description, status)
VALUES
('Health Camp', 'Organizing village health camps', 'Active'),
('Skill Development', 'Providing vocational training', 'Active');
```

User Database Configuration File

```
<?php
// Database connection settings
$host = "localhost";
$username = "root";
$password = ""; // Add password if set
$database = "gaonsetu_db";

// Create connection
$conn = new mysqli($host, $username, $password, $database);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
```

API for User Registration

```
<?php
include '../config/db_config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $phone = $_POST['phone'];
    $otp = rand(100000, 999999); // Generate random OTP

    // Insert or update user
    $query = "INSERT INTO users (name, phone, otp) VALUES ('$name', '$phone', '$otp')
            ON DUPLICATE KEY UPDATE otp='$otp'";

    if ($conn->query($query) === TRUE) {
        echo json_encode(["status" => "success", "message" => "OTP sent!", "otp" => $otp]);
    } else {
        echo json_encode(["status" => "error", "message" => "Error: " . $conn->error]);
    }
}
?>
```

API for User Registration

```
<?php
include '../config/db_config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $phone = $_POST['phone'];
    $otp = rand(100000, 999999); // Generate random OTP

    // Insert or update user
    $query = "INSERT INTO users (name, phone, otp) VALUES ('$name', '$phone', '$otp')
            ON DUPLICATE KEY UPDATE otp='$otp'";
```

```
    if ($conn->query($query) === TRUE) {
        echo json_encode(["status" => "success", "message" => "OTP sent!", "otp" => $otp]);
    } else {
        echo json_encode(["status" => "error", "message" => "Error: " . $conn->error]);
    }
}
?>
```

// 1. Database Configuration - config/db_config.php

```
<?php
$host = "localhost";
$username = "root";
$password = ""; // Add password if needed
$database = "gaonsetu_db";

$conn = new mysqli($host, $username, $password, $database);

if ($conn->connect_error) {ss
    die("Connection failed: " . $conn->connect_error);
}
?>
```

```
// 2. User Registration with OTP - api/user_register.php
<?php
include '../config/db_config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $name = $_POST['name'];
    $phone = $_POST['phone'];
    $otp = rand(100000, 999999);

    $query = "INSERT INTO users (name, phone, otp) VALUES ('$name', '$phone', '$otp')
            ON DUPLICATE KEY UPDATE otp='$otp'";
```

```
    if ($conn->query($query) === TRUE) {
        echo json_encode(["status" => "success", "message" => "OTP sent!", "otp" => $otp]);
    } else {
        echo json_encode(["status" => "error", "message" => "Error: " . $conn->error]);
    }
}
?>

// 3. OTP Verification - api/verify_otp.php
<?php
include '../config/db_config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $phone = $_POST['phone'];
    $otp = $_POST['otp'];

    $query = "SELECT * FROM users WHERE phone='$phone' AND otp='$otp'";
    $result = $conn->query($query);

    if ($result->num_rows > 0) {
        echo json_encode(["status" => "success", "message" => "OTP verified!"]);
    } else {
        echo json_encode(["status" => "error", "message" => "Invalid OTP!"]);
    }
}
?>

// 4. Save Language - api/save_language.php
<?php
include '../config/db_config.php';

if ($_SERVER['REQUEST_METHOD'] == 'POST') {
    $user_id = $_POST['user_id'];
    $language = $_POST['language'];
```

```
    $query = "INSERT INTO language_preferences (user_id, language_selected) VALUES ('$user_id',
'$language')
            ON DUPLICATE KEY UPDATE language_selected='$language'";

    if ($conn->query($query) === TRUE) {
        echo json_encode(["status" => "success", "message" => "Language saved!"]);
    } else {
        echo json_encode(["status" => "error", "message" => "Error: " . $conn->error]);
    }
}
?>

// 5. Admin View Data - api/admin_view.php
<?php
include '../config/db_config.php';

$query = "SELECT u.id, u.name, u.phone, l.language_selected, u.created_at
         FROM users u
         LEFT JOIN language_preferences l ON u.id = l.user_id";
$result = $conn->query($query);

$data = [];
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $data[] = $row;
    }
}

echo json_encode(["status" => "success", "data" => $data]);
?>
```

Api connection Admin Side

==========

```
<?php
// Database connection file - db.php

$host = '127.0.0.1';
$db_name = 'gaonsetu_db';
$username = 'root';
$password = ''; // Add your password if any

// Create a connection
$conn = new mysqli($host, $username, $password, $db_name);

// Check connection
if ($conn->connect_error) {
    die("Connection failed: " . $conn->connect_error);
}
?>
<?php
// Include database connection
include 'db.php';

// Fetch all villagers
$sql = "SELECT * FROM villagers";
$result = $conn->query($sql);

$villagers = [];
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $villagers[] = $row;
    }
    echo json_encode(['status' => 'success', 'data' => $villagers]);
} else {
```

```
    echo json_encode(['status' => 'error', 'message' => 'No villagers found']);
}
$conn->close();
?>
<?php
include 'db.php';

// Fetch all services
$sql = "SELECT * FROM services WHERE status = 'Active'";
$result = $conn->query($sql);

$services = [];
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $services[] = $row;
    }
    echo json_encode(['status' => 'success', 'data' => $services]);
} else {
    echo json_encode(['status' => 'error', 'message' => 'No active services found']);
}
$conn->close();
?>
<?php
include 'db.php';

// Fetch all requests
$sql = "SELECT r.request_id, v.name AS villager_name, s.service_name, r.status, r.created_at
        FROM requests r
        JOIN villagers v ON r.villager_id = v.villager_id
        JOIN services s ON r.service_id = s.service_id
        ORDER BY r.created_at DESC";

$result = $conn->query($sql);
```

```
$requests = [];
if ($result->num_rows > 0) {
    while ($row = $result->fetch_assoc()) {
        $requests[] = $row;
    }
    echo json_encode(['status' => 'success', 'data' => $requests]);
} else {
    echo json_encode(['status' => 'error', 'message' => 'No requests found']);
}
$conn->close();
?>

<?php
include 'db.php';

// Get data from POST request
$data = json_decode(file_get_contents('php://input'), true);

$service_name = $data['service_name'];
$description = $data['description'];

// Insert into services table
$sql = "INSERT INTO services (service_name, description) VALUES ('$service_name',
'$description')";
if ($conn->query($sql) === TRUE) {
    echo json_encode(['status' => 'success', 'message' => 'Service added successfully']);
} else
 {
    echo json_encode(['status' => 'error', 'message' => 'Error adding service: ' . $conn->error]);
}
$conn->close();
?>
<?php
include 'db.php';
```

```
// Get data from POST request
$data = json_decode(file_get_contents('php://input'), true);

$request_id = $data['request_id'];
$status = $data['status'];

// Update request status
$sql = "UPDATE requests SET status = '$status' WHERE request_id = $request_id";

if ($conn->query($sql) === TRUE) {
   echo json_encode(['status' => 'success', 'message' => 'Request updated successfully']);
}
Else
 {
   echo json_encode(['status' => 'error', 'message' => 'Error updating request: ' . $conn->error]);
}
$conn->close();
?>
<?php
include 'db.php';

// Get data from POST request
$data = json_decode(file_get_contents('php://input'), true);

$service_id = $data['service_id'];

// Delete service
$sql = "DELETE FROM services WHERE service_id = $service_id";
if ($conn->query($sql) === TRUE)
{
   echo json_encode(['status' => 'success', 'message' => 'Service deleted successfully']);
}
Else
```

```
 {
    echo json_encode(['status' => 'error', 'message' => 'Error deleting service: ' . $conn->error]);
}
$conn->close();
?>
```

5.2.1 Code Efficiency

1. Modular Design and Code Reusability

Approach:

- Divided the application into small, independent modules (e.g., Login, OTP Verification, Language Selection).
- Used modular coding principles to avoid redundancy and improve maintainability.

Benefit:

- Enhances code readability, reduces complexity, and promotes reusability.

2. API Call Optimization

Approach:

- Cached frequently accessed data to minimize repetitive API calls.
- Implemented API request throttling to avoid excessive server hits.

Benefit:

- Reduces latency and improves app performance by minimizing network usage.

3. Memory Management

Approach:

- Released unused resources and used Garbage Collection effectively to prevent memory leaks.
- Applied onDestroy() and onPause() methods to clean up resources when the app is not in use.

Benefit:

- Avoids memory bloat and enhances overall app stability.

4. Efficient UI Rendering

Approach:

- Used ConstraintLayout to optimize the UI structure.
- Minimized nested layouts to avoid layout inflation.

Benefit:

- Reduced UI rendering time and improved overall user experience.

5. Efficient Data Handling

Approach:

- Used SQLite and SharedPreferences to manage user preferences and store data locally.
- Optimized database queries with proper indexing and reduced redundant data operations.

Benefit:

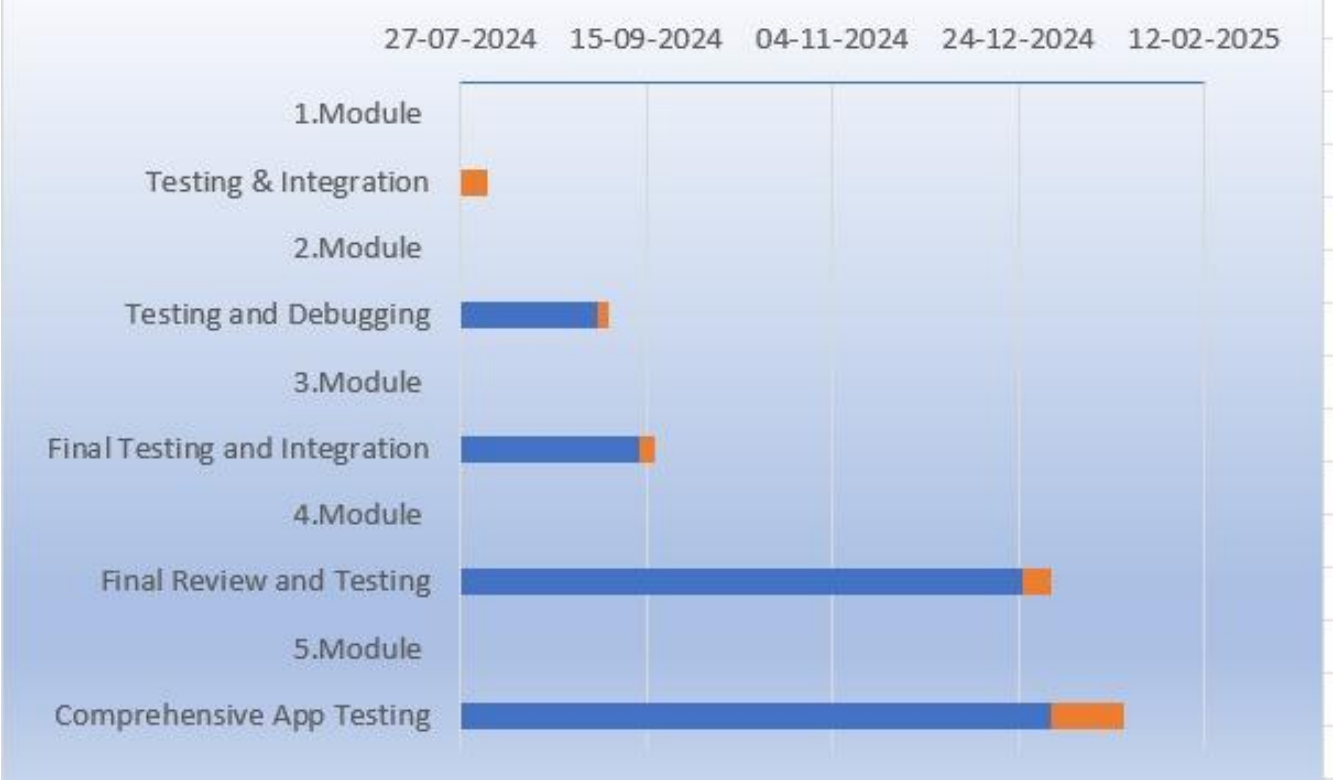
- Ensures faster data retrieval and improves application responsiveness.

5.3 Testing Approach

1.. Testing Model: Hybrid Approach

We are using a combination of **Category Partitioning** and **State Machine-Based Testing** models to ensure robust testing.

2.Testing Phases and Timeline



5.3Testing Gantt Chart

3.Test Case Design

Test cases designed based on functional and non-functional requirements.

Covers:

- Positive and negative scenarios.
- Boundary value analysis.
- Equivalence partitioning.

5.3.1 Unit Testing

1. Functional Unit Testing

- **Goal:** Test individual functions/methods for correctness.
- **Approach:**
  - Validate language selection logic using SharedPreferences.
  - Test OTP generation, verification, and timeout scenarios.
  - Ensure sign-up, login, and admin-side operations work as expected.
  - Test CRUD operations (Create, Read, Update, Delete) in MySQL.

---

2. UI Unit Testing

- **Goal:** Verify UI interactions and behavior.
- **Approach:**
  - Check the functionality of buttons, checkboxes, and list items.
  - Validate correct navigation between screens:
    - Language selection → Home
    - Login/Sign-up → Dashboard
  - Ensure the leaf-shaped buttons with checkboxes allow only one language selection.
- **Tools:**
  - Espresso for UI automation testing.
  - Mockito to mock dependencies in UI interactions.

---

3. Database Unit Testing (MySQL)

- **Goal:** Test data consistency and integrity.
- **Approach:**
  - Validate successful insertion, retrieval, and deletion of user data.
  - Test that the selected language preference is stored and retrieved correctly.
  - Verify admin-side CRUD operations for managing village data.
- **Tools:**
  - Use Mockito to mock database queries.
  - Room or direct MySQL integration testing with test data.

---

4. API Unit Testing

- **Goal:** Test the backend API functionality.
- **Approach:**

    - Test API endpoints for:
        - OTP generation and verification.
        - User sign-up and login.
        - Fetching and updating user preferences.
    - Handle edge cases such as invalid OTP, expired sessions, and server errors.
- **Tools:**
    - Retrofit for API calls.
    - Mockito and OkHttp to mock API responses.

---

5. SharedPreferences Unit Testing

- **Goal:** Verify that language preferences are stored and retrieved correctly.
- **Approach:**
    - Test saving and fetching language preference.
    - Ensure the correct language is applied throughout the app.
    - Check that the language selection screen only shows once during the first install.
- **Tools:**
    - Robolectric to simulate Android context.
    - Mockito for mocking SharedPreferences.

---

6. OTP-Based Authentication Testing

- **Goal:** Test OTP functionality and security.
- **Approach:**
    - Validate OTP generation and correctness.
    - Test auto-verification and manual entry workflows.
    - Handle incorrect OTP and retry scenarios gracefully.

5.3.2 Integrated Testing

Integrated testing ensures that different modules and components in the **GaonSetu** app work seamlessly together. Since GaonSetu has multiple interacting layers (UI, backend APIs, database, etc.), testing their interaction is essential to identify inconsistencies and ensure smooth functioning.

- **Goal:** Test the interaction between multiple modules, APIs, database, and user interface.
- **Scope:**
  - Language selection and application across the app.
  - OTP-based login and verification workflows.
  - Database interaction and data persistence.
  - Admin side functionalities for managing village data.

5.3.3 Beta Testing

Beta Testing is essential to ensure that the GaonSetu app is ready for real-world usage, especially since the app targets villagers and aims to provide an easy-to-use interface with language selection, OTP-based login, and admin-side functionality.

- **Goal:** Gather real-world feedback by allowing a select group of users (beta testers) to use the app.
- **Purpose:** Identify bugs, improve app usability, and validate features under real usage scenarios.
- **Focus Areas:**
  - Language selection and consistency.
  - OTP-based login and verification.
  - Admin operations and data sync.
  - UI/UX experience and ease of use.

5.4 Modifications and Improvements

1. Language Selection Improvements

Modify:

- Ensure the selected language is stored in SharedPreferences only after confirming the choice.
- Add a confirmation popup: "Are you sure you want to proceed with [Selected Language]?" before saving the preference.
- Optimize backend to update language preferences in the MySQL database as well.

Improve:

- Allow users to preview the app in their chosen language before confirming the selection.
- Add an option in Settings to reset or change the language easily.

---

2. OTP-Based Login/Sign-Up Enhancements

Modify:

- Implement automatic OTP detection for easier user experience.
- Add a 30-60 second timer for OTP resend option with proper validation.
- Store verified mobile numbers securely in MySQL.

Improve:

- Add error messages for incorrect OTP or connection failure.
- Enable auto-filling of OTP from SMS if possible.

---

3. Admin Panel Enhancements (Mobile App)

Modify:

- Design a clean and responsive admin interface with essential controls.
- Add an analytics dashboard to monitor user activity and language preferences.

Improve:

- Enable push notifications for important updates.
- Add a user feedback section to collect suggestions for improvements.

---

4. Backend & Database Optimization

Modify:

- Optimize SQL queries to reduce load time.
- Ensure secure storage of user data and language preferences.

Improve:

- Implement database backup and recovery mechanisms.
- Add error handling to prevent database crashes.

---

5. Logo Animation Implementation

Modify:

- Use Java animation libraries to animate the village hut, modern buildings, and bridge in the GaonSetu logo.
- Add smooth entry and exit animations on the splash screen.

Improve:

- Implement fade-in/fade-out effects to make the animation visually appealing.

---

6. Notification & Alerts System

Modify:

- Add push notifications to inform users about updates and announcements.

Improve:

- Enable personalized notifications based on user preferences.
- Add a toggle in settings to manage notification preferences.

---

7. Performance Optimization

Modify:

- Optimize app load time by compressing images and minimizing API calls.

Improve:

- Implement caching to reduce unnecessary server requests.
- Add offline mode support for key features.

5.6 Test Cases

Test Case ID	GS_TC_001	Test Case Description	Verify successful login with valid credentials.		
Created By	Prapti Singh & Sneha Yadav	Reviewed By		Version	

QA Tester's Log	Verified that user is able to log in successfully with correct credentials.

Tester's Name	Prapti Singh & Sneha Yadav	Date Tested	18-Jan-2025	Test Case (Pass/Fail/Not Executed)	Pass

S #	Prerequisites:
1	Internet connection (Wi-Fi or mobile data)
2	Android device or emulator with minimum version 7.0+
3	GaonSetu app installed and configured properly
4	Valid mobile number for OTP-based login
5	Test database with sample user accounts
6	MySQL database and backend APIs set upcand running
7	Admin panel configured for managing app content
8	Language selection configured in the app

Test Scenario	Verify user registration, login, language selection, service requests, location access, offline data sync, admin panel, security,

Step #	Step Details	Expected Results	Actual Results	Pass / Fail / Not executed
1	Submit OTP received on the registered mobile. Attempt login with correct and incorrect OTPs.	User should register and log in successfully with valid credentials.	As Expected	Pass
2	Verify if language is applied and can be changed later.	Selected language should be applied correctly and can be updated from settings.	As Expected	Pass
3	Upload a valid file format and verify submission.	Form should submit successfully with valid data and file.	As expected	Pass
4	Verify if admin can manage user and service data.	Admin should have access to manage app content and users.	As expected	Pass
5	Check data encryption during transmission.	User data should be securely stored and transmitted.	As expected	Pass
6	Check design consistency on different devices.	UI should be responsive and user-friendly.	As expected	Pass

CHAPTER 6

RESULTS AND DISCUSSION

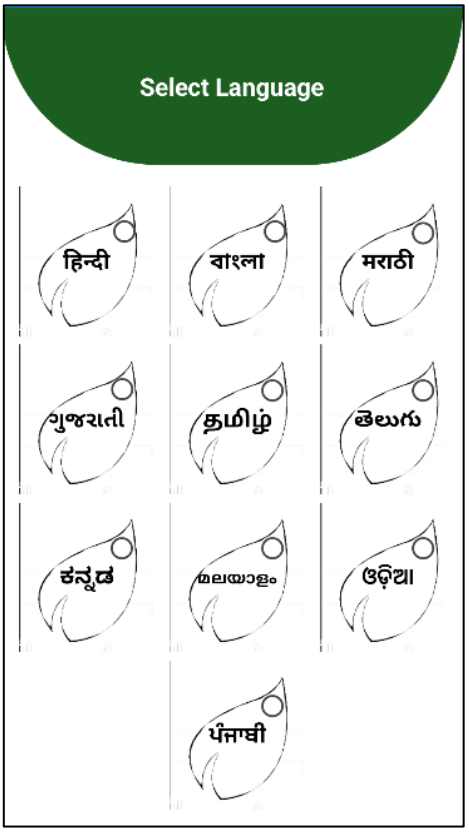
6.1 Test Reports

Test Scenario	Verify user registration, login, language selection, service requests, location access, offline data sync, admin panel, security,			
Step #	**Step Details**	**Expected Results**	**Actual Results**	**Pass / Fail / Not executed**
1	Submit OTP received on the registered mobile. Attempt login with correct and incorrect OTPs.	User should register and log in successfully with valid credentials.	As Expected	Pass
2	Verify if language is applied and can be changed later.	Selected language should be applied correctly and can be updated from settings.	As Expected	Pass
3	Upload a valid file format and verify submission.	Form should submit successfully with valid data and file.	As expected	Pass
4	Verify if admin can manage user and service data.	Admin should have access to manage app content and users.	As expected	Pass
5	Check data encryption during transmission.	User data should be securely stored and transmitted.	As expected	Pass
6	Check design consistency on different devices.	UI should be responsive and user-friendly.	As expected	Pass

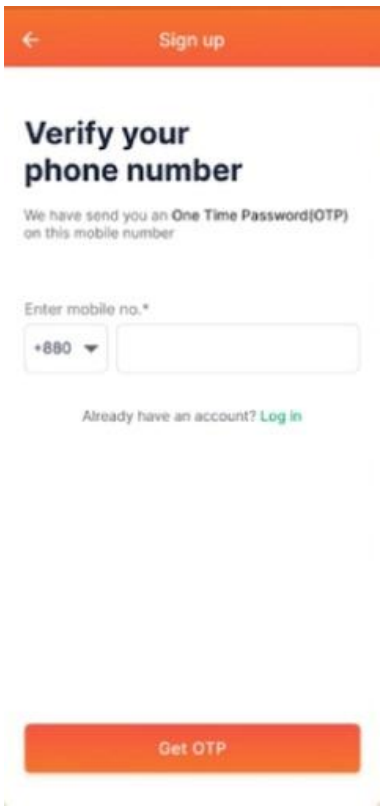
6.2 User Documentation

Splash Screen :-

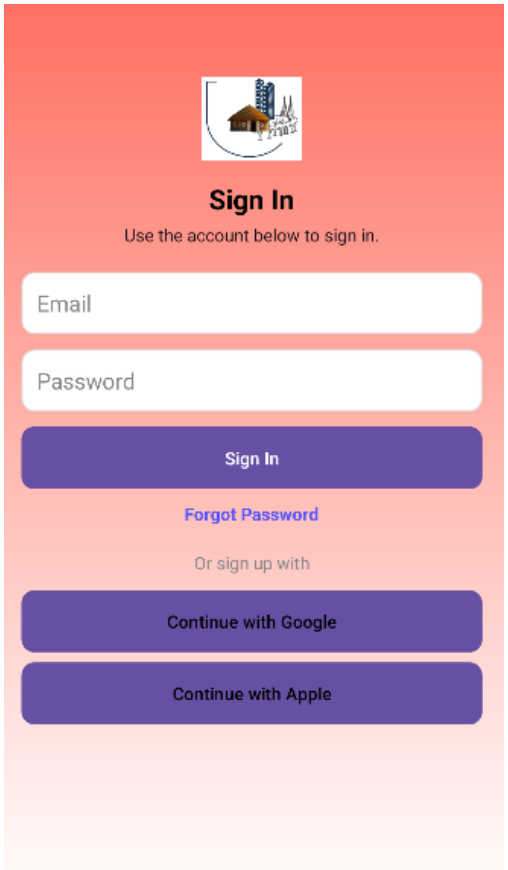
2.Language



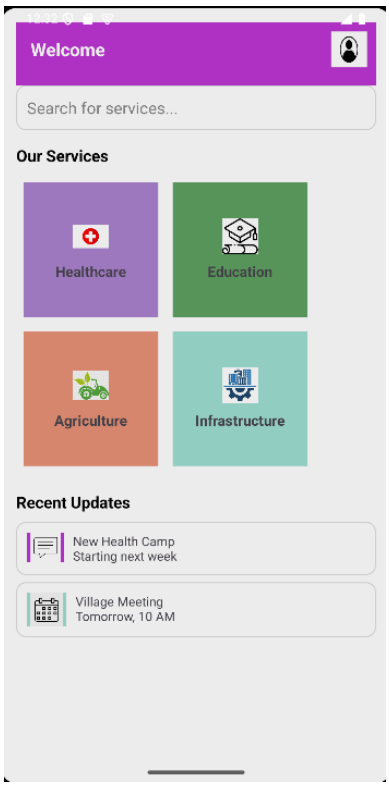
3.User Registration Page:



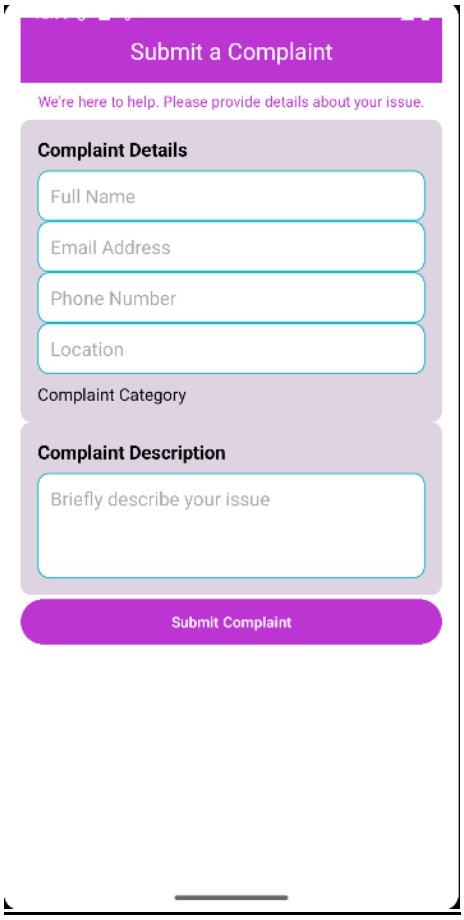
4.User Login Page



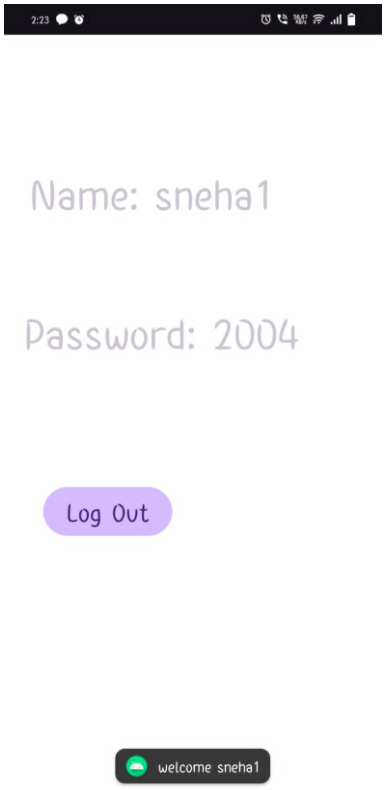
5.Home page



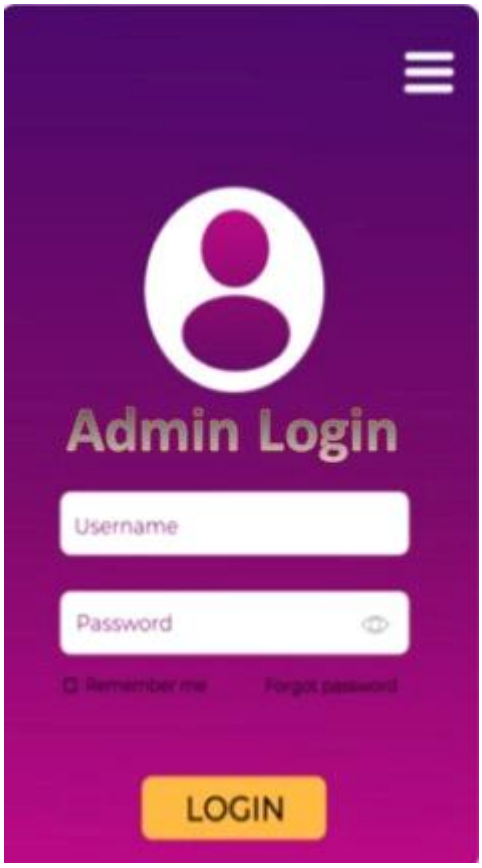
6.Complaint Page

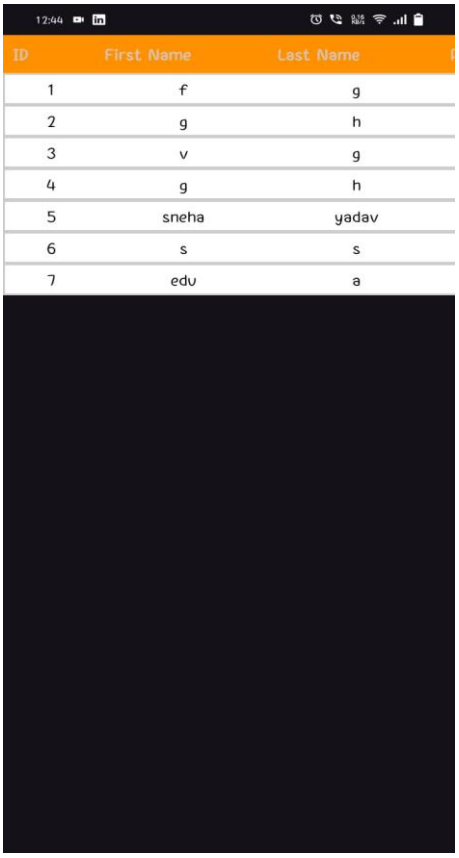


7.Profile Page



8.AdminPage:





ID	First Name	Last Name
1	f	g
2	g	h
3	v	g
4	g	h
5	sneha	yadav
6	s	s
7	edu	a

CHAPTER 7
CONCLUSIONS

7.1 Conclusion

The Gaon Setu App aims to bridge the digital gap between rural citizens and essential services by providing a centralized, user-friendly, and accessible mobile platform. Through detailed requirements gathering, system analysis, and careful planning, the app is designed to address real-world challenges like lack of awareness, poor service access, and limited digital literacy. By focusing on key features like service directories, grievance redressal, offline support, and multi-language accessibility—while ensuring data security and usability—the Gaon Setu App has the potential to empower rural communities and improve their quality of life through technology.

7.1.1 Significance of the System

The **Gaon Setu App** plays a vital role in enhancing rural development by acting as a digital bridge between villagers and essential services. It simplifies access to government schemes, health facilities, education, employment opportunities, and grievance redressal — all within a single, easy-to-use mobile application.

By offering features like local language support, offline functionality, and real-time updates, the system ensures inclusivity for users with low digital literacy and limited internet access. It empowers citizens with information, streamlines communication with authorities, and promotes transparency in public service delivery.

Ultimately, **Gaon Setu** contributes to digital empowerment, social inclusion, and improved governance in rural areas, aligning with the vision of **Digital India**.

7.2 Limitations of the System

While the **Gaon Setu App** provides valuable functionality for rural users, there are certain limitations that may affect its performance and reach:

1. **Limited Internet Connectivity**
   Many rural areas still suffer from poor or no internet access, which can hinder real-time updates and syncing, despite offline support.

2. **Low Digital Literacy**
   Users with little to no experience with smartphones may find it difficult to navigate the app without training or guidance.

3. **Device Compatibility**
   Older or low-end Android devices may experience lag or may not support some app features due to hardware limitations.

4. **Language Barriers**
   Although the app supports local languages, it may not cover all dialects or accommodate non-literate users completely.

5. **Dependence on Accurate Government Data**
   The app relies heavily on up-to-date information from government sources. Delays or inaccuracies in data can affect reliability.

6. **Security Threats**
   While basic security measures are implemented, advanced threats like data breaches or targeted attacks may still pose risks if not continually updated.

7.3 Future Scope of the Project

The **Gaon Setu App** has the potential to grow significantly and serve as a comprehensive rural development platform. Below are some possible future enhancements and expansions:

1. **Integration with e-Governance Portals**
   Direct integration with state and central government portals (like DigiLocker, UMANG) for real-time updates, application submissions, and digital document access.

2. **AI-Based Chatbot Support**
   Implementing a multilingual voice/text chatbot for helping users navigate services, answer FAQs, and assist illiterate users.

3. **Expansion to iOS Platform**
   Launching an iOS version to expand accessibility across more devices and users.

4. **Data Analytics Dashboard**
   Admin dashboards for real-time insights on usage patterns, grievance trends, and scheme popularity to aid policy-making.

5. **Voice-Enabled Navigation**
   Adding full voice-guided navigation and content reading features for visually impaired and non-literate users.

6. **Payment Gateway Integration**
   For future use in fee-based services, utility bill payments, or micro-loans through UPI or mobile wallets.

7. **Community Forum**
   Enabling peer-to-peer interaction among villagers for sharing local updates, success stories, or asking questions.

8. **Partnerships with NGOs and Local Bodies**
   Collaborating with non-governmental organizations to extend app features like education drives, health camps, and awareness programs.

CHAPTER 8

REFERENCE

8.1 Reference

- www.youtube.com
- www.google.com
- www.wikipedia.com
- ChatGpt